Parte 2 — Sistemas Embarcados com STM32G070RB (ARM Cortex-M0+)

Capítulo 1 — O núcleo ARM Cortex-M0+

Este capítulo apresenta o núcleo ARM Cortex-M0+ do STM32G070RB sob a ótica de firmware bare-metal. O foco está nos elementos que aparecem em qualquer projeto sério: registradores do core, mapa de memória, vector table, exceções, NVIC, SysTick, PRIMASK e técnicas de debug de baixo nível.

Ao longo do texto, a abordagem é sempre prática: primeiro o conceito, depois a leitura dos registradores relevantes e, por fim, exemplos que podem ser reproduzidos no STM32CubeIDE. Sempre que houver diferença importante entre Cortex-M0+ e núcleos mais ricos, como M3 e M4, isso será destacado explicitamente.

---

1) O que exatamente é o Cortex-M0+

O STM32G070RB usa um Arm Cortex-M0+, arquitetura ARMv6-M, rodando até 64 MHz na família STM32G070 (conforme datasheet).

O M0+ executa instruções Thumb (sempre Thumb), com foco em simplicidade, baixo consumo e latência de interrupção muito baixa.

---

2) Registradores "visíveis" do núcleo

2.1 Registradores gerais R0–R12

- Registradores de propósito geral para cálculos, temporários e passagem de argumentos.
- Na convenção de chamada mais comum (AAPCS), R0–R3 carregam os primeiros argumentos e R0 costuma carregar o retorno (útil para ler assembly e entender custo de chamadas).

2.2 SP (Stack Pointer): MSP e PSP

O Cortex-M tem dois stack pointers:

- MSP (Main Stack Pointer): usado no reset e tipicamente em handlers.
- PSP (Process Stack Pointer): muito usado quando existe RTOS (thread mode usa PSP, handlers usam MSP).

2.3 LR (R14) e PC (R15)

- LR guarda retorno de subrotina. Em exceções, LR recebe um valor especial (EXC_RETURN) para retorno de exceção.
- PC aponta para a próxima instrução.

2.4 xPSR (Program Status Register)

O xPSR inclui:

- flags (N, Z, C, V)
- IPSR (Interrupt Program Status Register): número da exceção atual (excelente para debug de "onde estou").

---

3) Registradores especiais que você realmente usa em firmware

3.1 CONTROL

Controla (no thread mode):

- seleção de stack (MSP/PSP)
- privilegiado vs não-privilegiado

Em bare-metal, normalmente fica no padrão e você quase não mexe; em RTOS isso vira assunto central.

3.2 PRIMASK (chave de "trava global" das IRQs)

PRIMASK permite mascarar interrupções "normais" (mascáraveis).

No mundo real, ele é o mecanismo por trás de __disable_irq() e __enable_irq() (CMSIS). O PM0223 discute o uso de CPSIE/CPSID e intrinsics para isso.

✅ **Padrão profissional:**

- salvar PRIMASK atual
- desabilitar IRQ
- executar trecho crítico
- restaurar PRIMASK original (não "forçar enable")

---

4) Mapa de memória do ARM (visão "core-first")

4.1 As grandes regiões (modelo Cortex-M)

O ARM define um mapa “padrão” de regiões (código, SRAM, periféricos e a área do próprio core). O manual ARMv6-M descreve o System Address Map e o bloco do System Control Space (SCS).

4.2 System Control Space (SCS) / PPB

A região onde vivem os periféricos do próprio core (NVIC, SysTick, SCB e afins) fica no espaço de sistema (SCS).

No PM0223 você vê endereços como 0xE000ED08 (VTOR), 0xE000ED0C (AIRCR) etc.

Isso é importante porque: NVIC/SysTick/SCB não são periféricos da ST; eles são parte do “core-side”.

---

5) Tabela de vetores (Vector Table)

5.1 O que é

A tabela de vetores contém:

- valor inicial do stack (MSP) no endereço 0x00000000
- endereço do Reset_Handler
- endereços dos handlers de exceção (NMI, HardFault…)
- endereços das IRQs externas

O PM0223 deixa claro que:

- no reset, a vector table está fixa em 0x00000000
- o bit menos significativo do endereço do handler deve ser 1 (Thumb).

5.2 VTOR (Vector Table Offset Register)

No M0+ usado pela ST, existe VTOR em 0xE000ED08, para realocar a tabela de vetores.

E o PM0223 recomenda usar barreira após atualizar VTOR (ex.: DMB) para garantir que a nova tabela seja usada em exceções subsequentes.

---

6) Exceções e prioridades “do core”

6.1 Quem são as exceções

No Cortex-M0+ (visão do PM0223):

- Reset (prioridade mais alta)
- NMI

- HardFault (fault handler principal)
- SVCall
- PendSV
- SysTick
- IRQs externas (via NVIC)

E um ponto crucial: no M0+ HardFault é o único fault handler (não existe aquela separação rica de MemManage/BusFault/UsageFault como em M3/M4).

6.2 Prioridades "fixas" de Reset/NMI/HardFault

O PM0223 lista prioridades especiais:

- Reset: prioridade -3 (a mais alta)
- NMI: -2
- HardFault: -1

Essas são exceções do sistema; o restante (SysTick, PendSV, SVCall e IRQs externas) é configurável.

---

7) NVIC (Nested Vectored Interrupt Controller)

7.1 O que o NVIC controla

- enable/disable da IRQ
- pending/clear pending
- prioridade
- estado active (quando a IRQ está sendo executada)
- aninhamento (preempção) conforme prioridade

7.2 Registradores principais do NVIC (core-side)

Os registradores NVIC típicos incluem:

- ISER/ICER (set/clear enable)
- ISPR/ICPR (set/clear pending)
- IPR (prioridade)

O PM0223 detalha os Interrupt Priority Registers NVIC_IPR0..NVIC_IPR7, com um campo de 8 bits por IRQ, 4 IRQs por registrador.

7.3 Quantos bits de prioridade existem de verdade?

Um ponto importante, e frequentemente fonte de erro, e a quantidade real de bits de prioridade implementados.

No STM32 Cortex-M0+ descrito pelo PM0223:

- o campo tem 8 bits, mas o processador implementa apenas bits [7:6] (os bits [5:0] lêem como zero e ignoram escrita).

✅ **Consequência prática:**

- você tem poucos níveis reais de prioridade (2 bits → 4 níveis efetivos)
- escrever "255" vira "192" (porque só os bits altos contam).

Isso impacta diretamente como você organiza prioridades no firmware.

---

8) SysTick (timer do core)

SysTick é um periférico do core para gerar ticks periódicos (muito usado por HAL/RTOS).

No modelo ARMv6-M, SysTick entra no mesmo sistema de exceções; e o ARM manual discute o uso de barreiras para garantir correção ao habilitar/desabilitar exceções e mudanças em registradores do SCS.

Pontos que você precisa dominar:

- SysTick gera uma exceção (SysTick_Handler)
- prioridade de SysTick é configurável (system handler priority register)
- SysTick é frequentemente a base de "tempo do sistema"

---

9) SCB (System Control Block) — o painel de controle do core

No PM0223, alguns registradores do bloco do sistema aparecem explicitamente com endereço:

- VTOR: 0xE000ED08
- AIRCR (Application Interrupt and Reset Control Register): 0xE000ED0C
- SCR (System Control Register): 0xE000ED10
- CCR (Configuration and Control Register): 0xE000ED14
- SHPR2 / SHPR3 (prioridade de system handlers): 0xE000ED1C e 0xE000ED20

E o ARMv6-M manual descreve as regras de correção envolvendo SCS e barreiras (DSB/ISB) quando você precisa garantir que mudanças em registradores do sistema foram efetivadas.

---

10) Barramentos internos: Flash/RAM/periféricos (visão útil para firmware)

Sem entrar nos blocos ST (GPIO/timers), a visão que importa do core é:

- o core busca instruções e acessa dados através do sistema de barramento do MCU
- Flash e SRAM estão conectadas de forma que:
  - .text (código) geralmente está em Flash
  - .data/.bss/stack ficam em SRAM
- periféricos do core (NVIC/SysTick/SCB) ficam no espaço SCS

O ARMv6-M descreve o modelo de memória e a necessidade de barreiras para "correção do sistema" em eventos de configuração (por exemplo, desabilitar SysTick/NVIC de forma determinística e garantir que o estado não muda após a sequência correta com barreiras).

---

11) Endianness e "pegadinhas" de baixo nível

No Cortex-M, o padrão é little-endian para dados (e fetch de instrução é fixo). O PM0223 discute endianness e exemplos.

Isso afeta:

- empacotamento de dados em protocolos
- uso de union para bytes/palavras
- interoperabilidade com "network order"

---

12) Checklist "núcleo dominado"

Quando o leitor terminar este capítulo, ele deve conseguir:

- listar e explicar R0–R12, SP (MSP/PSP), LR, PC, xPSR/IPSR
- entender PRIMASK e implementar lock/unlock correto
- explicar a vector table e por que VTOR existe (e como usar barreira)

- entender NVIC: enable/pending/priority e os bits reais de prioridade (bits [7:6])
- entender SysTick como exceção do core
- entender por que no M0+ HardFault é o “fault” principal
- reconhecer o espaço SCS (0xE000EDxx) e os registradores principais

---

Recomendo compilar em Debug com -Og (ou -O0) para facilitar o passo-a-passo e evitar que variáveis “sumam” por otimização.

Preparação do projeto

Passo A — criar projeto

1. **File → New → STM32 Project**
2. **Selecione STM32G070RB (ou a Nucleo correspondente, se for o seu caso)**
3. **Escolha STM32CubeIDE + HAL (padrão)**
4. **Gere o projeto**

A gente não vai usar periféricos ST (GPIO/UART/TIM). Mas pode manter o projeto HAL gerado (SystemClock + startup). Só evite HAL_Delay() depois que reconfigurarmos SysTick manualmente.

Passo B — adicionar um arquivo para os exemplos

Crie CoreArmExamples.c e CoreArmExamples.h em Core/Src e Core/Inc.

Arquivos do capítulo (cole exatamente assim)

CoreArmExamples.h

```
#ifndef CORE_ARM_EXAMPLES_H

#define CORE_ARM_EXAMPLES_H


#include <stdint.h>


/* Execute os exemplos em sequência chamando core_examples_run(); */

void core_examples_run(void);
```

```
/* Variáveis globais “observáveis” no debugger (Watch/Live Expressions) */
extern volatile uint32_t g_ex1_snapshot_ready;

extern volatile uint32_t g_systick_ms;
extern volatile uint32_t g_irq_test_counter;

extern volatile uint32_t g_usart1_irq_count;
extern volatile uint32_t g_systick_irq_count;

/* Buffer para registrar ordem de execução (ex.: preempção) */
extern volatile uint8_t  g_trace[32];
extern volatile uint32_t g_trace_len;

#endif /* CORE_ARM_EXAMPLES_H */
```

CoreArmExamples.c

```
#include "CoreArmExamples.h"
#include "stm32g0xx.h"     /* CMSIS + definição de IRQn */
#include "core_cm0plus.h"  /* intrinsics Cortex-M0+ */

/* ============================================================
 * CONFIG: habilite/desabilite exemplos
 * ============================================================ */
#define RUN_EX1_READ_CORE_REGS         1
#define RUN_EX2_CRITICAL_SECTION       1
#define RUN_EX3_VTOR_RELOCATE_TO_RAM    1
#define RUN_EX4_SYSTICK_BARE           1
```

```
#define RUN_EX5_NVIC_PRIORITY_PENDING   1

/* ============================================================
 * GLOBAIS para observar no debugger
 * ============================================================ */
volatile uint32_t g_ex1_snapshot_ready = 0;

volatile uint32_t g_systick_ms = 0;
volatile uint32_t g_irq_test_counter = 0;

volatile uint32_t g_usart1_irq_count = 0;
volatile uint32_t g_systick_irq_count = 0;

volatile uint8_t  g_trace[32];
volatile uint32_t g_trace_len = 0;

/* ============================================================
 * Helpers de trace (para ver ordem de execução)
 * ============================================================ */
static void trace_push(uint8_t code)
{
   if (g_trace_len < sizeof(g_trace))
      g_trace[g_trace_len++] = code;
}

/* ============================================================
 * EX1) Ler registradores do core (IPSR/xPSR/PRIMASK/CONTROL/MSP/PSP)
 * ============================================================ */
```

```
typedef struct
{
    uint32_t ipsr;
    uint32_t xpsr;
    uint32_t primask;
    uint32_t control;
    uint32_t msp;
    uint32_t psp;
} core_snapshot_t;

static volatile core_snapshot_t g_snap;

static void ex1_read_core_regs(void)
{
    /* Capture “onde o core está” neste instante.
     * Coloque breakpoint aqui e observe g_snap no debugger.
     */
    g_snap.ipsr    = __get_IPSR();     /* 0 = Thread mode, >0 = exceção atual */
    g_snap.xpsr    = __get_xPSR();     /* flags + IPSR embutido */
    g_snap.primask = __get_PRIMASK();  /* 0 = IRQs on, 1 = IRQs masked */
    g_snap.control = __get_CONTROL();  /* MSP/PSP + privilégio (thread mode) */
    g_snap.msp     = __get_MSP();
    g_snap.psp     = __get_PSP();

    g_ex1_snapshot_ready = 1; /* só para o leitor saber que foi atualizado */
}

/* ============================================================
```

```
 * EX2) Seção crítica com PRIMASK (provar “perda” sem lock)
 *
 * Estratégia:
 * - Usaremos uma IRQ externa (USART1_IRQn) SEM configurar USART.
 * - Vamos disparar a IRQ por software usando NVIC_SetPendingIRQ.
 * - A ISR incrementa g_irq_test_counter.
 * - No main, fazemos um RMW (read-modify-write) e forçamos a IRQ no meio.
 *   Sem lock: pode perder incremento (condição de corrida).
 *   Com lock: fica consistente.
 * ============================================================ */

static inline uint32_t irq_lock(void)
{
    uint32_t key = __get_PRIMASK();
    __disable_irq();
    __DMB(); /* boa prática: impede reordenação ao redor da região crítica */
    return key;
}

static inline void irq_unlock(uint32_t key)
{
    __DMB();
    __set_PRIMASK(key);
}

void USART1_IRQHandler(void)
{
    /* Não tocamos em registradores do USART (periférico ST).
```

```
     * Estamos usando apenas o fato do NVIC chamar o handler quando pendente.
     */
    g_usart1_irq_count++;
    g_irq_test_counter++;       /* “recurso compartilhado” */
    trace_push(0xU1);           /* marca que IRQ rodou (0xU1 é só um código) */
}

static void ex2_critical_section_demo(void)
{
    /* Habilita a IRQ no NVIC (apenas NVIC, sem periférico). */
    NVIC_ClearPendingIRQ(USART1_IRQn);
    NVIC_EnableIRQ(USART1_IRQn);

    g_irq_test_counter = 0;
    g_usart1_irq_count = 0;

    /* ------------- Caso A: SEM lock (pode perder incremento) ------------- */
    for (int i = 0; i < 1000; i++)
    {
        uint32_t tmp = g_irq_test_counter;     /* leitura */
        NVIC_SetPendingIRQ(USART1_IRQn);       /* força ISR acontecer aqui */
        tmp++;                                 /* modifica */
        g_irq_test_counter = tmp;              /* escrita (pode sobrescrever incremento da ISR)
*/
    }

    /* Coloque breakpoint aqui e anote:
     * - g_usart1_irq_count (quantas ISRs rodaram)
```

```
 * - g_irq_test_counter (resultado final)
 *
 * Esperado:
 * - g_usart1_irq_count ~ 1000
 * - g_irq_test_counter pode ser < 2000 (perdas por corrida)
 */

/* ------------- Caso B: COM lock (não deve perder) ------------- */
g_irq_test_counter = 0;
g_usart1_irq_count = 0;

for (int i = 0; i < 1000; i++)
{
    uint32_t key = irq_lock();

    uint32_t tmp = g_irq_test_counter;
    NVIC_SetPendingIRQ(USART1_IRQn); /* enquanto IRQs estão desabilitadas, ela
vira pending */
    tmp++;
    g_irq_test_counter = tmp;

    irq_unlock(key);
    /* Ao reabilitar, a IRQ pending pode rodar logo em seguida */
}

/* Breakpoint aqui:
 * Esperado:
 * - g_usart1_irq_count ~ 1000
```

```
    * - g_irq_test_counter deve ficar muito próximo de 2000 (1000 main + 1000 ISR)
    */
}

/* ============================================================
 * EX3) Realocar Vector Table (VTOR) para RAM
 *
 * Por que isso é útil?
 * - Bootloader/app
 * - Hook de handlers em runtime
 * - Testes e instrumentação
 *
 * Procedimento:
 * 1) Copiar a tabela de vetores da Flash para um array na RAM
 * 2) Alterar uma entrada (ex.: USART1_IRQHandler) para um handler alternativo
 * 3) Apontar VTOR para a tabela na RAM
 * 4) Disparar IRQ e ver que o handler novo executa
 *
 * IMPORTANTE:
 * - O tamanho da vector table depende do MCU.
 * - Aqui usamos um tamanho “seguro” e explicamos como ajustar.
 * ============================================================ */

/* Ajuste se necessário: 16 system exceptions + N IRQs externas.
 * Para começar, 16 + 32 = 48 costuma cobrir muitos MCUs pequenos.
 * Se seu projeto tiver mais IRQs, aumente.
 */
#define VECTOR_COUNT  (48u)
```

```
static uint32_t g_vector_ram[VECTOR_COUNT] __attribute__((aligned(256)));

static void alt_usart1_handler(void)
{
    /* Handler alternativo: registra em trace */
    g_usart1_irq_count++;
    trace_push(0xA1); /* marcou que ALT handler rodou */
}

static void ex3_vtor_relocate_to_ram(void)
{
    /* 1) Copiar vetor atual (Flash) para RAM */
    uint32_t *vtor_src = (uint32_t *)SCB->VTOR; /* endereço atual */
    for (uint32_t i = 0; i < VECTOR_COUNT; i++)
        g_vector_ram[i] = vtor_src[i];

    /* 2) Substituir a entrada do USART1_IRQn */
    /* O índice na tabela = 16 + IRQn (porque primeiros 16 são system exceptions) */
    uint32_t idx = 16u + (uint32_t)USART1_IRQn;
    if (idx < VECTOR_COUNT)
    {
        /* Endereço do handler precisa ter bit0 = 1 (Thumb). */
        g_vector_ram[idx] = ((uint32_t)alt_usart1_handler) | 1u;
    }

    /* 3) Apontar VTOR para tabela na RAM */
    __DMB();
```

```
    SCB->VTOR = (uint32_t)g_vector_ram;
    __DMB();
    __ISB();

    /* 4) Testar: disparar pendência da IRQ */
    NVIC_ClearPendingIRQ(USART1_IRQn);
    NVIC_EnableIRQ(USART1_IRQn);
    NVIC_SetPendingIRQ(USART1_IRQn);

    /* Breakpoint aqui:
     * - Veja g_trace: deve conter 0xA1 (ALT) e não 0xU1 (original)
     * - Se não, aumente VECTOR_COUNT ou verifique o idx.
     */
}

/* ============================================================
 * EX4) SysTick “na unha” (sem HAL timebase)
 *
 * Atenção:
 * - HAL usa SysTick para HAL_Delay e timebase.
 * - Se você reconfigurar SysTick, evite HAL_Delay.
 *
 * Aqui vamos configurar SysTick para 1ms usando SystemCoreClock.
 * ============================================================ */

void SysTick_Handler(void)
{
    g_systick_irq_count++;
```

```
    g_systick_ms++;
    trace_push(0xS1);
}

static void systick_config_1ms(void)
{
    /* SysTick roda no clock do core (normalmente).
     * LOAD = (SystemCoreClock / 1000) - 1 para 1ms.
     */
    uint32_t ticks = (SystemCoreClock / 1000u) - 1u;

    SysTick->LOAD = ticks;
    SysTick->VAL  = 0u;

    /* CTRL:
     * ENABLE = 1
     * TICKINT = 1 (gera exceção)
     * CLKSOURCE = 1 (core clock)
     */
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk   |
                    SysTick_CTRL_ENABLE_Msk;
}

static void delay_ms_bare(uint32_t ms)
{
    uint32_t start = g_systick_ms;
    while ((g_systick_ms - start) < ms)
```

```
    {
        /* busy wait: didático */
    }
}

static void ex4_systick_bare_demo(void)
{
    g_systick_ms = 0;
    g_systick_irq_count = 0;

    systick_config_1ms();

    /* Breakpoint: veja g_systick_ms incrementando */
    delay_ms_bare(10);

    /* Depois do delay:
     * g_systick_ms ~ 10
     * g_systick_irq_count ~ 10
     */
}

/* ============================================================
 * EX5) NVIC: pending/enable + prioridades (bits reais no M0+)
 *
 * Vamos demonstrar:
 * - habilitar IRQ externa
 * - set pending por software
 * - setar prioridades e observar preempção
```

```
 *
 * No Cortex-M0+ (STM32G0), apenas os bits mais altos de prioridade
 * são implementados (tipicamente 2 bits efetivos).
 * ============================================================ */

static void ex5_nvic_priority_pending_demo(void)
{
    /* Limpa trace */
    g_trace_len = 0;

    /* Configura prioridades:
     * Em ARM, número menor = maior prioridade.
     * No M0+ efetivamente temos poucos níveis.
     *
     * Vamos tentar:
     * - USART1 IRQ como “mais alta”
     * - SysTick como “mais baixa”
     */
    NVIC_SetPriority(USART1_IRQn, 0);      /* mais alta */
    NVIC_SetPriority(SysTick_IRQn,  3);    /* mais baixa (dentro do possível) */

    /* Habilita */
    NVIC_ClearPendingIRQ(USART1_IRQn);
    NVIC_EnableIRQ(USART1_IRQn);

    /* Garante SysTick ativo (se o EX4 rodou antes, já está) */
    /* Se quiser garantir: systick_config_1ms(); */
```

```
    /* Dispara USART1 enquanto SysTick pode ocorrer */
    NVIC_SetPendingIRQ(USART1_IRQn);

    /* Breakpoint aqui:
     * - Veja g_trace: ordem dos códigos
     *   0xS1 = SysTick
     *   0xU1 = USART1 original
     *   0xA1 = USART1 alternativo (se EX3 fez swap)
     *
     * Se USART1 tiver prioridade mais alta, ele pode "entrar antes"
     * ou preemptar dependendo do momento.
     *
     * Observe também:
     * - NVIC_GetPriority(USART1_IRQn) e SysTick_IRQn
     * - Em M0+, apenas bits altos contam (valores podem parecer "quantizados")
     */
}

/* ============================================================
 * Runner: chama exemplos em ordem
 * ============================================================ */
void core_examples_run(void)
{
#if RUN_EX1_READ_CORE_REGS
    ex1_read_core_regs();
#endif

#if RUN_EX2_CRITICAL_SECTION
```

```
    ex2_critical_section_demo();
#endif

#if RUN_EX3_VTOR_RELOCATE_TO_RAM
    ex3_vtor_relocate_to_ram();
#endif

#if RUN_EX4_SYSTICK_BARE
    ex4_systick_bare_demo();
#endif

#if RUN_EX5_NVIC_PRIORITY_PENDING
    ex5_nvic_priority_pending_demo();
#endif

    /* Fim: coloque breakpoint aqui para inspecionar tudo */
    __NOP();
}
```

Chamada no main.c

No Core/Src/main.c, inclua e chame no começo do main() (depois do clock estar configurado):

```
#include "CoreArmExamples.h"

/* ... */

int main(void)
{
```

```
  HAL_Init();
  SystemClock_Config();

  /* NÃO use HAL_Delay depois do EX4 (SysTick manual), a menos que você
     entenda que reconfigurou o tick do HAL. */

  core_examples_run();

  while (1)
  {
  }
}
```

Como o leitor valida cada item (passo a passo no debugger)

Item 1 — Ler registradores do core

1. Coloque breakpoint em ex1_read_core_regs()
2. Abra Expressions / Live Expressions
3. Adicione:

o g_snap.ipsr → deve ser 0 no thread mode

o g_snap.primask → 0 normalmente

o g_snap.msp e g_snap.psp (PSP geralmente 0 se não usado)

4. Dê step e veja os valores mudarem

O que aprender aqui

- IPSR mostra se você está em thread/handler
- PRIMASK mostra se IRQs estão mascaradas
- MSP/PSP mostram qual stack está em uso

Item 2 — Região crítica (prova prática)

1. Coloque breakpoint entre os dois laços do EX2 (tem comentários “Caso A” e “Caso B”)

2. Inspecione:

o g_usart1_irq_count (quantas vezes a “ISR” rodou)

o g_irq_test_counter (valor final)

O que você deve ver

- Sem lock: g_irq_test_counter tende a ficar menor que 1000 (main) + 1000 (ISR)
- Com lock: g_irq_test_counter tende a ficar bem próximo de 2000

Item 3 — VTOR (vector table em RAM)

1. Coloque breakpoint no fim de ex3_vtor_relocate_to_ram()

2. Veja:

o SCB->VTOR apontando para g_vector_ram

o g_trace[] contendo 0xA1 (handler alternativo)

Se não funcionar

- aumente VECTOR_COUNT (o MCU pode ter mais IRQs que 32)
- verifique o índice: 16 + USART1_IRQn

Item 4 — SysTick “na unha”

1. Coloque breakpoint dentro de SysTick_Handler()

2. Veja g_systick_ms incrementando

3. Após delay_ms_bare(10):

o g_systick_ms ~ 10

Observação importante

- Evite HAL_Delay() depois disso, porque você “assumiu” o SysTick.

Item 5 — NVIC priorities/pending

1. Coloque breakpoint no fim de ex5_nvic_priority_pending_demo()

2. Inspecione:

o NVIC_GetPriority(USART1_IRQn)

o NVIC_GetPriority(SysTick_IRQn)

3. Observe g_trace para ver a ordem de execução

O que aprender aqui

- Você consegue disparar IRQ por software via pending
- Prioridade no M0+ é “quantizada” (poucos níveis reais)

A seguir está o guia de disassembly por item (EX1–EX5): o que procurar no assembly, por que aparece, e como isso prova que o core está fazendo o que a gente explicou. A ideia é o leitor abrir a janela Disassembly no STM32CubeIDE e conseguir “enxergar” o funcionamento do ARM.

Recomendo: Debug com -Og (ou -O0) para facilitar o mapeamento com o C. Em -O2/-Os muita coisa vira inline e some do jeito “óbvio”.

Como abrir o Disassembly no STM32CubeIDE

1. Inicie Debug (Run → Debug)
2. Abra Window → Show View → Disassembly
3. Coloque um breakpoint na linha desejada
4. Use Step Into (F5) e Step Over (F6) para acompanhar
5. Na mesma tela, observe:

o registradores (R0–R12, SP, LR, PC, xPSR)

o instruções LDR/STR, CMP, Bxx, PUSH/POP, BL

EX1 — Ler registradores do core (__get_IPSR, __get_xPSR, __get_PRIMASK, etc.)

O que observar no disassembly

As funções CMSIS de leitura de registradores normalmente viram instruções do tipo MRS (“Move from Special Register”):

- MRS rX, ipsr
- MRS rX, psr (ou xpsr dependendo da forma)

- MRS rX, primask
- MRS rX, control
- MRS rX, msp
- MRS rX, psp

Como isso aparece no fluxo

Você verá uma sequência:

1. MRS para ler o registrador especial para um registrador geral (ex.: r0)
2. STR para gravar o valor em memória (no g_snap)

✅ **Prova prática:**

- se o compilador gerou MRS primask, você tem certeza que o valor foi lido do core naquele momento (não é "variável de C").

O que o leitor aprende aqui

- registradores especiais do core são lidos via instruções especiais (não via memória comum)
- o debugger mostra exatamente os valores do core porque eles realmente existem na CPU

EX2 — Seção crítica com PRIMASK (race sem lock vs com lock)

Parte A: Sem lock (o problema)

No C (sem lock) existe um padrão perigoso:

- lê g_irq_test_counter (load)
- gera pending IRQ
- incrementa local
- escreve de volta (store)

O que procurar

Você vai ver:

- LDR do contador global
- incrementos em registradores (ADDS)
- STR gravando de volta

E, principalmente, você verá que nada impede que a ISR rode entre o LDR e o STR. No disassembly, isso não aparece como “trava”, porque realmente não existe.

✅ **Prova prática no debug:**

- Se você colocar breakpoint nessa região e disparar pending, verá que a ISR pode acontecer “no meio” do read-modify-write.

Parte B: Com lock (o conserto)

No irq_lock() e irq_unlock() você deve ver instruções de máscara global:

O que procurar (Cortex-M0+)

- CPSID i → desabilita IRQs (global mask)
- CPSIE i → habilita IRQs (se fosse “forçar enable”)
- ou (mais profissional) sequência com PRIMASK:

o MRS rX, primask (captura chave)

o CPSID i (desabilita)

o MSR primask, rX (restaura)

No nosso código, usamos __get_PRIMASK() e __set_PRIMASK(key):

- espere ver MRS ... primask e MSR primask, ...
- e também CPSID i (porque __disable_irq() normalmente vira isso)

Barreiras

Você pode ver algo como:

- DMB (data memory barrier)

✅ **Prova prática:**

- com PRIMASK=1, a ISR não entra “no meio” do read-modify-write
- a IRQ fica pending e roda depois que você destravar

O que o leitor aprende aqui

- “race condition” em firmware é literalmente “interromper no meio”
- PRIMASK resolve a corrida quando o recurso compartilhado é acessado por ISR
- o disassembly mostra exatamente a instrução que trava e destrava (cpsid, msr primask)

EX3 — VTOR e troca de handler (vector table em RAM)

O que observar no disassembly

1) Cópia do vetor (loop de memória)

Você verá um loop com:

- LDR de vtor_src[i]
- STR em g_vector_ram[i]
- ADDS para incrementar ponteiros/índices
- CMP / BNE para repetir

Isso prova que:

- o vetor realmente foi copiado para RAM

2) Escrever o novo handler no índice

Você deve ver:

- cálculo do índice idx = 16 + USART1_IRQn
- STR guardando o endereço alt_usart1_handler|1

Detalhe importantíssimo (Thumb):

- o endereço do handler tem bit0=1
- no disassembly, você pode ver um ORR com #1 antes de armazenar

3) Atualizar VTOR

Procure escrita em:

- SCB->VTOR

No disassembly isso vira:

- LDR do endereço base do SCB
- STR em offset do VTOR

E ao redor:

- DMB e ISB (barreiras)

✅ **Prova prática:**

- após VTOR apontar para RAM, o NVIC busca handlers da nova tabela

- ao setar pending de USART1, o handler alternativo executa (trace = 0xA1)

O que o leitor aprende aqui

- vector table é só um array de endereços
- VTOR muda “de onde” o core pega o endereço da ISR
- por isso bootloader/app e técnicas de hook funcionam

EX4 — SysTick “na unha” (timer do core)

O que observar no disassembly

1) Escritas em registradores SysTick

Os registradores do SysTick ficam no System Control Space (core-side).

Você verá sequências de:

- LDR de endereço base do SysTick
- STR para LOAD, VAL, CTRL

Procure no C:

SysTick->LOAD = ...

SysTick->VAL = ...

SysTick->CTRL = ...

No assembly, isso vira store em endereços fixos.

2) Entrada/saída do SysTick_Handler

Quando a ISR dispara, observe:

- prólogo: PUSH {…} (salva registradores usados)
- corpo: incrementa g_systick_ms
- epílogo: POP {…} e retorno de exceção

No Cortex-M, o retorno da exceção envolve o hardware desempilhar o frame automaticamente.

✅ **Prova prática:**

- o handler realmente é chamado porque você vê o PC entrar nele
- o contador sobe de forma periódica

O que o leitor aprende aqui

- SysTick é um periférico do core
- sua configuração é apenas escrita em registradores mapeados
- ISR tem custo (push/pop + stacking automático)

EX5 — NVIC: pending/enable + prioridade "quantizada" no M0+

O que observar no disassembly

1) NVIC_EnableIRQ / NVIC_SetPendingIRQ

Essas calls (CMSIS) viram escrita em registradores NVIC:

- ISER (enable set)
- ISPR (pending set)

No disassembly, você verá stores em endereços do NVIC no SCS.

Se o compilador inline, você verá algo como:

- LDR base NVIC
- STR com bit correspondente ao IRQn

2) NVIC_SetPriority

No M0+, a prioridade é escrita em registradores IPR[].

Você verá:

- cálculo do índice do IPR (4 IRQs por registrador)
- shift para posicionar o campo
- store do valor

E a parte mais importante:

- quando você lê de volta (NVIC_GetPriority) pode parecer que valores foram "arredondados"
- porque só alguns bits mais altos contam (no STM32 M0+ típico, efetivo é 2 bits)

✅ **Prova prática:**

- no Watch, você seta 0 e 3 e vê que internamente eles viram poucos níveis
- no disassembly, você vê a escrita no NVIC->IPR

3) Preempção (ordem no trace)

A preempção perfeita depende de timing, então o que você procura é:

- o trace mostrando que um handler pode ocorrer “entre” instruções do outro
- e que a prioridade influencia qual entra primeiro

O que o leitor aprende aqui

- NVIC é um bloco “core-side”, controlado por memória mapeada
- pending pode ser setado por software (ótimo para teste)
- prioridade no M0+ é limitada → planeje com poucos níveis

Dicas de depuração

1) “Por que meu código sumiu no disassembly?”

- otimização remove variáveis e inlines funções.
- use -Og para debug e volatile para valores que precisam existir.

2) Como provar que volatile está funcionando?

- coloque um loop que lê um volatile e compare com um não-volatile em -O2.
- no disassembly, o volatile continua gerando LDR repetido; o não-volatile pode ser mantido em registrador.

3) Onde ver stack usage em ISR?

- no disassembly, observe quantos registradores são PUSH/POP.
- lembre que existe stacking automático também (r0-r3,r12,lr,pc,xpsr).

Checklist final do leitor (após EX1–EX5)

- Sei identificar MRS/MSR, CPSID, DMB/DSB/ISB no disassembly.
- Sei provar uma race condition e corrigir com PRIMASK.
- Sei o que é VTOR e como trocar vector table para RAM.
- Sei configurar SysTick sem HAL e entender o custo do handler.
- Entendo NVIC enable/pending/priority e as limitações de prioridade no M0+.

SysTick — Visão geral

O SysTick é um timer down-counter de 24 bits (conta de LOAD até 0), que pode:

- gerar uma exceção SysTick (interrupção) quando chega em 0
- servir como base de tempo (ticks de 1ms, por exemplo)
- permitir delays, timeouts e “heartbeat” do sistema

Registradores (offsets):

- CTRL (0x000) — controle e status
- LOAD (0x004) — valor de recarga (RELOAD)
- VAL (0x008) — valor atual do contador
- CALIB (0x00C) — calibração / referência (somente leitura)

---

1) CTRL — SysTick Control and Status Register (R/W)

Bits principais (CMSIS)

Os nomes de bits normalmente aparecem no CMSIS como:

- SysTick_CTRL_ENABLE_Msk
- SysTick_CTRL_TICKINT_Msk
- SysTick_CTRL_CLKSOURCE_Msk
- SysTick_CTRL_COUNTFLAG_Msk

Bit 0 — ENABLE

- 0: SysTick parado (counter desabilitado)
- 1: SysTick contando (counter habilitado)

✅ **Efeito prático:**

- se ENABLE=0, VAL não vai decrementar (timer fica parado)

Bit 1 — TICKINT

- 0: SysTick não gera exceção/IRQ ao chegar em 0
- 1: SysTick gera a exceção SysTick_Handler ao chegar em 0

✅ **Efeito prático:**

- você pode usar SysTick só como “contador” (polling) ou como “tick por interrupção”.

Bit 2 — CLKSOURCE

Seleciona a fonte de clock do SysTick. No Cortex-M, existem duas opções conceituais:

- 0: clock externo de referência (quando disponível, muitas vezes AHB/8 ou similar, depende do chip/implementação)
- 1: clock do processador (core clock / AHB)

⚠️ **Observação prática (STM32):**

- Em muitos STM32, CLKSOURCE=1 significa usar HCLK (AHB clock).
- CLKSOURCE=0 costuma significar alguma referência dividida (muitas vezes HCLK/8 em famílias antigas), mas isso pode variar por implementação.

✅ **Regra para o leitor:**

- Para tick de 1ms “padrão”, use CLKSOURCE=1 e calcule LOAD com SystemCoreClock.

Bit 16 — COUNTFLAG (Read-only)

- Este bit é setado em 1 quando o contador chega a 0 (evento de “wrap”).
- Ele é limpo automaticamente quando você lê CTRL (ou seja: leitura “consome” o flag).

✅ **Efeito prático:**

- em modo polling, você pode detectar o tick lendo COUNTFLAG
- se você não ler CTRL, pode “perder” a noção de quantos wraps ocorreram (é só 1 bit)

Exemplo de polling:

```
if (SysTick->CTRL & SysTick_CTRL_COUNTFLAG_Msk)
{
  // aconteceu um wrap (chegou em 0 pelo menos uma vez)
}
```

⚠️ **Armadilha:**

- COUNTFLAG não conta quantos ticks passaram; ele só indica que ocorreu pelo menos um wrap desde a última leitura de CTRL.

---

2) LOAD — SysTick Reload Value Register (R/W)

Campo RELOAD (bits [23:0])

- SysTick é um contador de 24 bits, então o reload máximo é:

o 0x00FFFFFF (16.777.215)

Quando o contador chega em 0:

- ele recarrega automaticamente com o valor de LOAD
- e continua contando

✅ **Fórmula do período (conceitual):**

- periodo = (LOAD + 1) / f_clk

Para 1ms (modo comum)

Se CLKSOURCE=1 e f_clk = SystemCoreClock:

LOAD = (SystemCoreClock / 1000) - 1

⚠️ **Armadilhas importantes:**

1. LOAD não pode passar de 24 bits
2. se SystemCoreClock não estiver correto (clock mal configurado), o tick sai errado
3. LOAD=0 significa período de 1 ciclo (porque é LOAD+1) — raramente desejado

---

3) VAL — SysTick Current Value Register (R/W)

Campo CURRENT (bits [23:0])

- Lendo VAL, você obtém o valor atual do contador (down-counter).
- Escrevendo em VAL (qualquer valor), você zera o contador e o COUNTFLAG.

✅ **Uso prático:**

- Ao configurar SysTick, é comum fazer:

o SysTick->VAL = 0;

para limpar o contador e evitar um “primeiro período” estranho.

⚠️ **Importante:**

- VAL é o contador atual, não o reload.
- Se você mudar LOAD no meio do caminho, o efeito completo só aparece após o próximo wrap.

---

4) CALIB — SysTick Calibration Register (R/O)

O CALIB dá informações de calibração do SysTick. Em muitos chips, ele pode indicar:

- um valor de referência para 10ms (TENMS)
- se existe referência externa (NOREF)
- se é “skewed” (SKEW) — implementações onde o valor de calibração não é exato.

Campo típico: TENMS (bits [23:0])

- valor “calibrado” que representaria 10ms para uma fonte de referência específica

Bit SKEW

- indica que TENMS não é um valor exato

Bit NOREF

- indica ausência de clock de referência externa

✅ **Como usar em firmware**

Na prática, em STM32, a maioria dos projetos ignora CALIB e configura SysTick usando:

- SystemCoreClock
- LOAD = (SystemCoreClock/1000)-1 para 1ms

Mas é bom saber que CALIB existe para aplicações que querem uma referência “calibrada”.

Sequência recomendada de configuração (passo a passo)

Para evitar glitches:

1. Desabilitar SysTick

SysTick->CTRL = 0;

2. Configurar LOAD (24 bits)

SysTick->LOAD = (SystemCoreClock / 1000u) - 1u;

3. Limpar contador atual e flags

SysTick->VAL = 0;

4. Configurar CTRL

- CLKSOURCE (1 = core clock)
- TICKINT (1 = habilita SysTick_Handler)
- ENABLE (1 = começa a contar)

```
SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                SysTick_CTRL_TICKINT_Msk   |
                SysTick_CTRL_ENABLE_Msk;
```

Verificação no debugger (o que o leitor deve olhar)

- SysTick->LOAD deve conter o valor calculado
- SysTick->VAL deve diminuir com o tempo (se ENABLE=1)
- SysTick->CTRL deve ter:

o ENABLE=1

o TICKINT=1 (se você quer interrupção)

o CLKSOURCE=1 (normal)

- COUNTFLAG pisca quando ocorre wrap (se você ler CTRL repetidamente)

Exemplos práticos curtos

A) SysTick por interrupção (tick de 1ms)

```
volatile uint32_t g_ms = 0;


void SysTick_Handler(void)
{
   g_ms++;
```

```
}
```

B) SysTick por polling (sem interrupção)

Configura TICKINT=0, e no loop:

```
if (SysTick->CTRL & SysTick_CTRL_COUNTFLAG_Msk)
{
   // ocorreu um tick
}
```

Erros clássicos (e como evitar)

1. Configurar SysTick e depois usar HAL_Delay

- HAL depende do SysTick configurado do jeito que ele espera.
- Se você reconfigurar, HAL_Delay() pode quebrar.

2. LOAD fora de 24 bits

- se passar de 0x00FFFFFF, SysTick não funciona corretamente.

3. SystemCoreClock errado

- tick fica errado (ms não é ms).

4. Esquecer SysTick->VAL = 0

- pode dar “primeiro período” encurtado/estranho.

5. Achar que COUNTFLAG conta ticks

- ele só indica “ocorreu wrap desde a última leitura”.

O aprofundamento natural deste assunto inclui:

1. Tabela completa bit a bit (CTRL/LOAD/VAL/CALIB) com máscaras e posições.
2. Um exemplo replicável no STM32CubeIDE que:

o configura SysTick sem HAL

o mede o tick no debugger

o mostra COUNTFLAG em polling

o mostra prioridade do SysTick no NVIC/SCB (core-side)

Parte A — SysTick: tabela bit a bit (formal)

Referência de nomes: CMSIS (core_cm0plus.h). SysTick no Cortex-M0+ é um contador de 24 bits.

1) SysTick->CTRL (Offset 0x000) — Control and Status (R/W)

Bit	Nome (CMSIS)	Acesso	Função
0	ENABLE	R/W	Habilita o contador. 1 = conta; 0 = parado.
1	TICKINT	R/W	Interrupção SysTick. 1 = gera exceção ao wrap; 0 = sem IRQ (polling).
2	CLKSOURCE	R/W	Seleciona fonte de clock. 1 = clock do core (HCLK); 0 = referência externa/alternativa (depende do chip).
15:3	—	R	Reservado (leitura como 0).
16	COUNTFLAG	R	Flag “wrap ocorreu”. Fica 1 quando contou até 0. Limpa ao ler CTRL.
31:17	—	R	Reservado.

Observações profissionais

- COUNTFLAG não é contador: é um “evento aconteceu desde a última leitura”.
- O comportamento do CLKSOURCE=0 pode variar; em STM32 costuma ser uma fonte derivada (ex.: HCLK/8 em algumas famílias), mas para o livro use como regra: use CLKSOURCE=1 para tick de tempo.

---

2) SysTick->LOAD (Offset 0x004) — Reload Value (R/W)

Bits	Nome	Acesso	Função
23:0	RELOAD	R/W	Valor recarregado quando o contador chega a 0. Máximo: 0x00FFFFFF.
31:24	—	R	Reservado.

Observações profissionais

- Período (em ciclos) = RELOAD + 1
- Se RELOAD for maior que 24 bits, o resultado é incorreto (não confiável).
- Para 1ms: RELOAD = (SystemCoreClock/1000) - 1

---

3) SysTick->VAL (Offset 0x008) — Current Value (R/W)

Bits	Nome	Acesso	Função
23:0	CURRENT	R/W	Valor atual do contador (down-counter).
31:24	—	R	Reservado.

Observações profissionais

- Escrever qualquer valor em VAL zera o contador e normalmente limpa COUNTFLAG.

(Por isso usamos SysTick->VAL = 0; na configuração.)

- VAL é útil para medir "quanto falta" até o próximo tick (debug).

---

4) SysTick->CALIB (Offset 0x00C) — Calibration (R/O)

Bit(s)	Nome	Acesso	Função
23:0	TENMS	R	Valor calibrado para 10ms (quando aplicável).
24	SKEW	R	1 = TENMS não é exato (calibração imperfeita).
25	NOREF	R	1 = não há referência externa.
31:26	—	R	Reservado.

Observações profissionais

- Em STM32, é comum ignorar CALIB e usar SystemCoreClock.
- CALIB é útil em sistemas que querem referência padrão de tempo (quando disponível e confiável).

Sequência "padrão ouro" de configuração

```
/* 1) Para o SysTick antes de mexer */

SysTick->CTRL = 0;


/* 2) Define o reload (24 bits) para 1ms */

SysTick->LOAD = (SystemCoreClock / 1000u) - 1u;
```

```
/* 3) Zera contador e limpa flags */
SysTick->VAL = 0u;

/* 4) Liga: core clock + interrupção + enable */
SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
            SysTick_CTRL_TICKINT_Msk   |
            SysTick_CTRL_ENABLE_Msk;
```

---

1) SCB->CPUID (Offset 0x000) — CPUID Base Register (R/O)

Esse registrador identifica:

- implementador (ARM)
- variante
- parte (qual core)
- revisão

Campos típicos (padrão ARM):

- Implementer [31:24]
- Variant [23:20]
- Architecture [19:16]
- PartNo [15:4]
- Revision [3:0]

✅ **Para Cortex-M0+, o PartNo identifica o núcleo (é um valor padrão ARM).**

Na prática, para o leitor:

- usamos CPUID para confirmar que estamos em um core “M0+” e ver revisão.

Exemplo de leitura (debug):

```
volatile uint32_t cpuid = SCB->CPUID;
```

Como validar sem UART/GPIO: use breakpoints + Watch/Live Expressions + Disassembly.

---

1) SCB->ICSR (Offset 0x004) — Interrupt Control and State Register (R/W)

O ICSR é o “painel” do core para:

- saber qual exceção/IRQ está ativa
- saber qual exceção/IRQ está pendente
- setar/limpar pendência de SysTick e PendSV
- setar pendência de NMI por software

No Cortex-M0+ (ARMv6-M), os bits mais usados são estes:

1.1 Tabela bit a bit (ICSR)

Bit(s)	Nome	Acesso	Descrição prática
8:0	VECTACTIVE	R	Número da exceção/IRQ atualmente em execução. 0 = Thread mode.
11	RETTOBASE	R	1 indica que ao terminar a exceção atual, o core retorna para o “base level” (não há outra exceção pendente a retornar). Útil para entender nesting.
20:12	VECTPENDING	R	Número da exceção/IRQ pendente que será atendida (se existir).
22	ISRPENDING	R	1 se existe alguma interrupção pendente no NVIC.
25	PENDSTCLR	W1	Escreva 1 para limpar pendência do SysTick.
26	PENDSTSET	W1	Escreva 1 para setar pendência do SysTick (forçar SysTick_Handler).
27	PENDSVCLR	W1	Escreva 1 para limpar pendência do PendSV.
28	PENDSVSET	W1	Escreva 1 para setar pendência do PendSV (forçar PendSV_Handler).
31	NMIPENDSET	W1	Escreva 1 para pendurar NMI (gera NMI_Handler). Use com cuidado.

Observações importantes

- Bits PEND*SET/CLR e NMIPENDSET são write-1-to-set/clear: escrever 1 executa a ação.

- VECTACTIVE e VECTPENDING retornam números de exceção, não “nomes”. Mesmo assim, é ótimo para debug porque você sabe se está em Thread/SysTick/PendSV/HardFault etc.
- PendSV é muito usado por RTOS (context switch), mas aqui vamos usar como “exceção de teste”.

---

2) Exemplos de código para testar o ICSR (passo a passo)

2.1 Arquivo scb_icsr_examples.h

Crie em Core/Inc:

```
#ifndef SCB_ICSR_EXAMPLES_H

#define SCB_ICSR_EXAMPLES_H


#include <stdint.h>


void scb_icsr_examples_run(void);


/* Variáveis para observar no debugger */

extern volatile uint32_t g_vectactive;

extern volatile uint32_t g_vectpending;

extern volatile uint32_t g_isrpending;

extern volatile uint32_t g_rettobase;


extern volatile uint32_t g_systick_count;

extern volatile uint32_t g_pendsv_count;


#endif
```

2.2 Arquivo scb_icsr_examples.c

Crie em Core/Src:

```
#include "scb_icsr_examples.h"
```

```
#include "stm32g0xx.h"
#include "core_cm0plus.h"

/* Observáveis */
volatile uint32_t g_vectactive = 0;
volatile uint32_t g_vectpending = 0;
volatile uint32_t g_isrpending = 0;
volatile uint32_t g_rettobase = 0;

volatile uint32_t g_systick_count = 0;
volatile uint32_t g_pendsv_count = 0;

/* Helpers: extrai campos do ICSR */
static void snap_icsr(void)
{
    uint32_t icsr = SCB->ICSR;

    g_vectactive  = (icsr & 0x1FFu);            /* bits 8:0 */
    g_rettobase   = (icsr >> 11) & 0x1u;        /* bit 11 */
    g_vectpending = (icsr >> 12) & 0x1FFu;      /* bits 20:12 */
    g_isrpending  = (icsr >> 22) & 0x1u;        /* bit 22 */
}

/* Handlers (core-side) */
void SysTick_Handler(void)
{
    g_systick_count++;
    snap_icsr(); /* snap dentro do handler: VECTACTIVE deve mudar */
```

```
}

void PendSV_Handler(void)
{
    g_pendsv_count++;
    snap_icsr();
}

static void force_systick_pending(void)
{
    /* Força exceção SysTick via ICSR (PENDSTSET) */
    SCB->ICSR = (1u << 26);
}

static void clear_systick_pending(void)
{
    /* Limpa pendência SysTick via ICSR (PENDSTCLR) */
    SCB->ICSR = (1u << 25);
}

static void force_pendsv_pending(void)
{
    /* Força exceção PendSV via ICSR (PENDSVSET) */
    SCB->ICSR = (1u << 28);
}

static void clear_pendsv_pending(void)
{
```

```
    /* Limpa pendência PendSV via ICSR (PENDSVCLR) */
    SCB->ICSR = (1u << 27);
}

void scb_icsr_examples_run(void)
{
    /* 1) Snapshot no Thread mode (esperado VECTACTIVE=0) */
    snap_icsr();
    __NOP(); /* breakpoint A: veja g_vectactive/g_vectpending */

    /* 2) Forçar PendSV por software (sem periférico ST) */
    clear_pendsv_pending();
    force_pendsv_pending();
    __NOP(); /* breakpoint B: PendSV deve ter rodado (g_pendsv_count++) */

    /* 3) Forçar SysTick por software
     * Observação: isso chama SysTick_Handler mesmo sem configurar SysTick timer.
     */
    clear_systick_pending();
    force_systick_pending();
    __NOP(); /* breakpoint C: SysTick deve ter rodado (g_systick_count++) */

    /* 4) Ver estados finais */
    snap_icsr();
    __NOP(); /* breakpoint D */
}
```

Como o leitor testa (passo a passo)

1. No main.c, chame scb_icsr_examples_run(); depois do clock (SystemClock_Config()).

2. Rode em Debug.

3. Coloque breakpoint nos __NOP() (A/B/C/D).

4. Observe no Watch:

o Em A: g_vectactive deve ser 0 (Thread mode).

o Em B: g_pendsv_count deve aumentar e dentro do handler g_vectactive terá valor ≠ 0.

o Em C: g_systick_count deve aumentar.

o g_vectpending e g_isrpending ajudam a ver se havia pendências.

Isso é uma prova prática de que ICSR é o "controle/estado" das exceções do core.

---

3) SCB->VTOR (Offset 0x008) — Vector Table Offset Register (R/W)

O VTOR define onde está a tabela de vetores (endereços de handlers). Isso é essencial para:

- bootloader + aplicação
- atualizar handlers em RAM
- instrumentação/testes

3.1 Campo e alinhamento

- VTOR contém o endereço base da vector table.
- O endereço precisa ser alinhado.

Para Cortex-M0+, a regra prática segura é alinhar em 256 bytes (deixar bits [7:0] = 0).

(Alguns casos aceitam 128, mas usar 256 deixa o exemplo robusto.)

✅ **Regra do livro:**

Sempre alinhe a tabela a 256 bytes e use __attribute__((aligned(256))).

---

4) Exemplo de código: realocar a vector table para RAM (VTOR)

4.1 scb_vtor_example.c (cole em Core/Src)

Este exemplo:

1. copia a tabela atual (Flash) para um array em RAM
2. troca PendSV_Handler por um handler alternativo
3. grava SCB->VTOR apontando para RAM
4. força PendSV via ICSR para provar que o handler mudou

```
#include "stm32g0xx.h"
#include "core_cm0plus.h"
#include <stdint.h>

/* Ajuste conforme necessário.
 * 16 exceções de sistema + N IRQs externas.
 * Para teste do PendSV/SysTick, 48 costuma ser suficiente.
 */
#define VECTOR_COUNT 48u

static uint32_t g_vector_ram[VECTOR_COUNT] __attribute__((aligned(256)));

/* Observáveis */
volatile uint32_t g_alt_pendsv_hits = 0;

static void alt_PendSV_Handler(void)
{
   g_alt_pendsv_hits++;
}

/* função de teste */
void scb_vtor_relocate_demo(void)
{
   uint32_t *src = (uint32_t *)SCB->VTOR;
```

```
/* 1) copia vetor da Flash para RAM */
for (uint32_t i = 0; i < VECTOR_COUNT; i++)
   g_vector_ram[i] = src[i];

/* 2) troca PendSV handler (PendSV é exceção do sistema)
 * Índice PendSV na tabela:
 * 0 = MSP, 1 = Reset, ..., PendSV fica no slot padrão do Cortex-M.
 * No CMSIS, PendSV é exceção do sistema, não IRQ externa.
 * O slot do PendSV fica na área dos system handlers.
 *
 * Para evitar "decorar número", a forma didática é:
 * - usar o fato que o projeto já tem PendSV_Handler
 * - e substituir diretamente o endereço do slot PendSV
 *
 * A posição do PendSV no vetor padrão é 14 (a partir do Reset=1) ou
 * depende do layout do startup; em STM32 costuma ser fixo:
 *  - 0: MSP
 *  - 1: Reset
 *  - ...
 *  - 14: PendSV
 *  - 15: SysTick
 *
 * Aqui vamos assumir o layout padrão Cortex-M:
 * PendSV = slot 14.
 */
const uint32_t PENDSV_SLOT = 14u;
if (PENDSV_SLOT < VECTOR_COUNT)
```

```
    g_vector_ram[PENDSV_SLOT] = ((uint32_t)alt_PendSV_Handler) | 1u;

  /* 3) grava VTOR */
  __DMB();
  SCB->VTOR = (uint32_t)g_vector_ram;
  __DMB();
  __ISB();

  /* 4) força PendSV via ICSR para provar */
  SCB->ICSR = (1u << 27); /* clear pending */
  SCB->ICSR = (1u << 28); /* set pending -> deve chamar alt handler */

  __NOP(); /* coloque breakpoint: g_alt_pendsv_hits deve ter incrementado */
}
```

Como o leitor testa

1. Chame scb_vtor_relocate_demo(); no main() (após clock).
2. Rode em Debug, pare no __NOP().
3. Veja:

o SCB->VTOR apontando para g_vector_ram

o g_alt_pendsv_hits maior que 0

Se não funcionar: aumente VECTOR_COUNT e confirme se o slot do PendSV no seu startup é o padrão (em STM32 geralmente é).

---

1) SCB->AIRCR (Offset 0x00C) — Application Interrupt and Reset Control (R/W)

O AIRCR é usado para:

- solicitar reset do sistema por software
- (em cores maiores) configurar agrupamento de prioridade

No M0+, o uso mais comum e realmente útil é SYSRESETREQ.

1.1 Campos (AIRCR)

Bits	Nome	Acesso	Descrição prática
31:16	VECTKEY	W	"Chave" obrigatória para escrever no AIRCR. Deve ser 0x5FA para aceitar escrita.
15	ENDIANNESS	R	Indica endianness (Cortex-M é little-endian em STM32).
10:8	PRIGROUP	R/W*	Agrupamento de prioridade (mais relevante em M3/M4). No M0+ é limitado e geralmente ignorado/sem efeito prático.
2	SYSRESETREQ	R/W	Se escrito com chave correta, pede reset do sistema.
1	VECTCLRACTIVE	R/W*	Limpa estado ativo de exceções (uso raro; cuidado).
0	VECTRESET	R/W*	Reset do core (uso legado/raríssimo; não use).

* Alguns campos podem existir no registro, mas seu uso é restrito e/ou não recomendado em M0+.

Nota essencial: VECTKEY

Para escrever no AIRCR, você precisa escrever:

- VECTKEY = 0x5FA no campo [31:16]

Exemplo de escrita "correta":

```
SCB->AIRCR = (0x5FAu << 16) | (1u << 2); // SYSRESETREQ
```

1.2 Exemplo replicável: reset por software

✅ **O leitor deve testar com cuidado (isso reseta mesmo).**

```
static void scb_system_reset(void)
{
  __DSB(); /* garante que tudo antes terminou */
  SCB->AIRCR = (0x5FAu << 16) | (1u << 2); /* SYSRESETREQ */
  __DSB();
  while (1) { __NOP(); } /* espera reset */
}
```

Como testar:

- coloque breakpoint antes do SCB->AIRCR = ...

- rode, depois dê “Resume”
- o MCU reinicia e volta ao main

---

2) SCB->SCR (Offset 0x010) — System Control Register (R/W)

O SCR controla comportamento de sono (sleep) e eventos:

- se WFI/WFE entram em Sleep ou Deep Sleep
- se a saída do sono depende de eventos ou interrupções

2.1 Campos (SCR)

Bit	Nome	Acesso	Descrição prática
4	SEVONPEND	R/W	1 = pendência de exceção gera “evento” (WFE pode acordar com pending).
2	SLEEPDEEP	R/W	1 = WFI/WFE entra em deep sleep (STOP/STANDBY dependem do MCU).
1	SLEEPONEXIT	R/W	1 = ao sair de ISR, volta direto para sleep (útil em sistemas event-driven).

Observações (firmware)

- SLEEPONEXIT é poderoso para sistemas “interrupt-driven”, mas pode confundir iniciante: o MCU parece “preso dormindo” se não houver interrupções.
- SLEEPDEEP só muda o “tipo de sono” do core; o modo deep real depende do controlador de energia do MCU (isso é periférico ST e fica para depois).

2.2 Exemplo replicável: SleepOnExit (sem periféricos ST)

Você pode testar a mecânica com SysTick (core-side) acordando o core:

1. Configure SysTick para 1ms (como já fizemos)
2. Habilite SLEEPONEXIT
3. No loop principal, execute __WFI()

```
static void scb_sleep_on_exit_enable(void)
{
  SCB->SCR |= (1u << 1); /* SLEEPONEXIT */
}
```

```
static void scb_sleep_on_exit_disable(void)
{
   SCB->SCR &= ~(1u << 1);
}
```

Como testar:

- habilite SysTick com interrupção
- habilite SLEEPONEXIT
- coloque breakpoint no SysTick_Handler e veja que o core “vive” indo e voltando do sleep

---

3) SCB->CCR (Offset 0x014) — Configuration and Control Register (R/W)

No M0+ o CCR é bem mais simples que em M3/M4. Ele controla alguns comportamentos do core.

3.1 Campos típicos (CCR)

Bit	Nome	Acesso	Descrição prática
9	STKALIGN	R/W	Alinhamento de stack em 8 bytes em entrada de exceção (boa prática ABI).
3	UNALIGN_TRP	R/W*	Trap em acesso desalinhado (em M0+ pode ser limitado/depende).
1	USERSETMPEND	R/W*	Permite set pending de SysTick/PendSV em modo não privilegiado (relevante com privilege separation).

* Alguns bits podem não ter o mesmo efeito que em cores maiores. O mais importante para firmware típico é STKALIGN.

STKALIGN (bit 9)

- Quando 1, o core mantém stack alinhada a 8 bytes ao entrar em exceções, o que melhora compatibilidade ABI e evita problemas com algumas rotinas (especialmente com toolchain/RTOS).

✅ **Recomendação: manter STKALIGN habilitado (muitos start-ups já deixam assim).**

3.2 Exemplo replicável: ler CCR e confirmar STKALIGN

```
volatile uint32_t g_ccr;
```

```
static void scb_ccr_snapshot(void)
{
   g_ccr = SCB->CCR;
}
```

Como testar:

- chame scb_ccr_snapshot()
- observe g_ccr no debugger
- veja se bit 9 está 1

---

4) SCB->SHP[2] (Offset 0x01C) — System Handler Priority Registers (R/W)

No Cortex-M0+, prioridades de system handlers ficam em SHP:

- SHP[0] é reservado (no layout mostrado)
- SHP cobre prioridades de:

o SVCall

o PendSV

o SysTick

✅ **Isso é core-side e muito importante para RTOS e para organizar preempção.**

4.1 Como as prioridades funcionam no M0+

Mesmo que o campo seja 8 bits, apenas os bits mais altos são implementados (tipicamente 2 bits efetivos no STM32G0). Então:

- valores reais possíveis são "quantizados" (ex.: 0, 64, 128, 192), dependendo do shift.

4.2 Exemplo replicável: setar prioridade de SysTick e PendSV

CMSIS oferece:

```
NVIC_SetPriority(SysTick_IRQn, prio);
NVIC_SetPriority(PendSV_IRQn,  prio);
NVIC_GetPriority(...)
```

Teste:

```
volatile uint32_t g_prio_systick;
```

```
volatile uint32_t g_prio_pendsv;


static void scb_set_system_priorities(void)
{
   NVIC_SetPriority(SysTick_IRQn, 3);
   NVIC_SetPriority(PendSV_IRQn,  0);


   g_prio_systick = NVIC_GetPriority(SysTick_IRQn);
   g_prio_pendsv  = NVIC_GetPriority(PendSV_IRQn);
}
```

Como testar:

- chame scb_set_system_priorities()
- observe que os valores lidos podem não ser exatamente 0 e 3, e sim quantizados (por causa dos bits implementados)

---

5) SCB->SHCSR (Offset 0x024) — System Handler Control and State Register (R/W)

⚠️ **Em cores maiores (M3/M4), SHCSR controla enable/pending/active de vários faults (MemManage/BusFault/UsageFault) e handlers.**

No Cortex-M0+, esses faults não existem separados — a maioria converge em HardFault. Então SHCSR tem utilidade bem menor que em M3/M4.

Mesmo assim, ele existe para:

- indicar estados de system handlers (dependendo da implementação)
- e manter compatibilidade de programação

5.1 Campos (visão prática)

Em muitos materiais, SHCSR inclui bits como:

- SVCALLACT, PENDSVACT, SYSTICKACT, etc. (actives)
- SVCALLPENDED, PENDSVSET, SYSTICKPENDED (pendings)

No M0+, a forma mais confiável de observar pending/active no dia a dia é:

- SCB->ICSR (VECTACTIVE/VECTPENDING + PENDSTSET/PENDSVSET)

- NVIC-> registers para IRQ externas
- use SHCSR mais como “registro de estado compatível”, e prefira ICSR para o que realmente importa no M0+.

5.2 Exemplo replicável: snapshot do SHCSR

```
volatile uint32_t g_shcsr;


static void scb_shcsr_snapshot(void)
{
   g_shcsr = SCB->SHCSR;
}
```

Como testar:

- chame scb_shcsr_snapshot() em thread mode e dentro de um handler (ex.: SysTick_Handler)
- compare valores no debugger (podem indicar bits de active/pending dependendo do core)

Código “runner” para testar tudo (AIRCR→SHCSR)

Crie scb_more_examples.c e chame no main().

```
#include "stm32g0xx.h"
#include "core_cm0plus.h"
#include <stdint.h>


volatile uint32_t g_aircr;
volatile uint32_t g_scr;
volatile uint32_t g_ccr;
volatile uint32_t g_shp0;
volatile uint32_t g_shp1;
volatile uint32_t g_shcsr;
```

```
volatile uint32_t g_prio_systick;
volatile uint32_t g_prio_pendsv;

static void scb_snap_all(void)
{
   g_aircr = SCB->AIRCR;
   g_scr   = SCB->SCR;
   g_ccr   = SCB->CCR;
   g_shp0  = SCB->SHP[0];
   g_shp1  = SCB->SHP[1];
   g_shcsr = SCB->SHCSR;
}

static void scb_set_system_priorities_demo(void)
{
   NVIC_SetPriority(SysTick_IRQn, 3);
   NVIC_SetPriority(PendSV_IRQn,  0);

   g_prio_systick = NVIC_GetPriority(SysTick_IRQn);
   g_prio_pendsv  = NVIC_GetPriority(PendSV_IRQn);
}

void scb_aircr_to_shcsr_demo(void)
{
   /* Snapshot inicial */
   scb_snap_all();
   __NOP(); /* breakpoint A */
```

```
/* SCR: SLEEPONEXIT on/off (apenas demonstrar bit) */
SCB->SCR |= (1u << 1);
scb_snap_all();
__NOP(); /* breakpoint B */

SCB->SCR &= ~(1u << 1);
scb_snap_all();
__NOP(); /* breakpoint C */

/* CCR snapshot */
scb_snap_all();
__NOP(); /* breakpoint D */

/* SHP: prioridades de SysTick/PendSV */
scb_set_system_priorities_demo();
scb_snap_all();
__NOP(); /* breakpoint E */

/* AIRCR reset (NÃO execute sem querer)
 * Se o leitor quiser testar, descomente com cuidado:
 */
// __DSB();
// SCB->AIRCR = (0x5FAu << 16) | (1u << 2);
// __DSB();
// while (1) { __NOP(); }

__NOP(); /* breakpoint F */
```

}

Como testar:

1. Chame scb_aircr_to_shcsr_demo(); no main() após clock.
2. Rode em debug.
3. Use breakpoints A…F e observe:

o g_scr mudando quando set/clear SLEEPONEXIT

o g_prio_systick/g_prio_pendsv mostrando valores quantizados

o snapshots de AIRCR/CCR/SHP/SHCSR

NVIC — Visão geral (o que ele controla)

O NVIC (Nested Vectored Interrupt Controller) controla as IRQs externas do microcontrolador (as interrupções de periféricos e eventos externos). Ele mantém para cada IRQ:

- Enable (habilitada ou não)
- Pending (pendente: requisitada e aguardando execução)
- Priority (prioridade)
- (Active existe internamente, mas não aparece nesses registradores no M0+ via essa struct simplificada)

✅ **Em firmware, os 4 comandos que você mais usa são:**

- Enable/Disable IRQ
- Set/Clear pending
- Set/Get priority
- Entender preempção (aninhamento) via prioridades

---

1) NVIC->ISER[0] — Interrupt Set-Enable Register (Offset 0x000)

O que faz

- Cada bit habilita uma IRQ.
- Escrever 1 no bit n habilita a IRQ n.

- Escrever 0 não faz nada (não desabilita).

Bit a bit (ISER[0])

- Bits [0..31]: SETENA[n]

o 1: IRQ n habilitada

o 0: IRQ n desabilitada

✅ **Leitura:**

- ler ISER mostra quais IRQs estão habilitadas.

✅ **Escrita típica:**

NVIC->ISER[0] = (1u << irqn);

⚠️ **Cuidado:**

- Só funciona para IRQn de 0 a 31 (no M0+ muitas MCUs têm menos que isso; no CMSIS do M0+ aparece como [1U] justamente por suportar até 32).

---

2) NVIC->ICER[0] — Interrupt Clear-Enable Register (Offset 0x080)

O que faz

- Cada bit desabilita uma IRQ.
- Escrever 1 no bit n desabilita a IRQ n.
- Escrever 0 não faz nada.

Bit a bit (ICER[0])

- Bits [0..31]: CLRENA[n]

o 1: desabilita IRQ n

✅ **Escrita típica:**

NVIC->ICER[0] = (1u << irqn);

---

3) NVIC->ISPR[0] — Interrupt Set-Pending Register (Offset 0x100)

O que faz

- Força uma IRQ a ficar pendente (como se o periférico tivesse pedido).
- Escrever 1 no bit n seta pendência para IRQ n.

Bit a bit (ISPR[0])

- Bits [0..31]: SETPEND[n]

o 1: IRQ n fica pending

✅ **Uso profissional:**

- testar handlers sem precisar do periférico real
- “deferir” trabalho: marcar pending e deixar o NVIC chamar

✅ **Escrita típica:**

NVIC->ISPR[0] = (1u << irqn);

---

4) NVIC->ICPR[0] — Interrupt Clear-Pending Register (Offset 0x180)

O que faz

- Limpa a pendência de uma IRQ.
- Escrever 1 no bit n limpa o pending.

Bit a bit (ICPR[0])

- Bits [0..31]: CLRPEND[n]

o 1: limpa IRQ n pending

✅ **Escrita típica:**

NVIC->ICPR[0] = (1u << irqn);

---

5) NVIC->IP[0..7] — Interrupt Priority Registers (Offset 0x300)

O que faz

Cada IRQ tem 1 byte de “prioridade”:

- IP[i] é um registrador de 32 bits no CMSIS, mas conceitualmente ele contém 4 prioridades de 8 bits (em muitos CMSIS ele aparece como array de 8 words para cobrir até 32 IRQs).

No Cortex-M0+ (e no STM32G0), a regra prática é:

✅ **Somente os bits mais altos da prioridade são implementados (tipicamente 2 bits efetivos).**

Isso significa:

- você escreve 0..255, mas só “degraus” (4 níveis) realmente importam.

Como interpretar

- menor número = maior prioridade (mais urgente)
- maior número = menor prioridade

Quantização (exemplo)

Se só bits [7:6] valem, os níveis efetivos são:

- 0b00xxxxxx → prioridade “0”
- 0b01xxxxxx → prioridade “1”
- 0b10xxxxxx → prioridade “2”
- 0b11xxxxxx → prioridade “3”

Então valores típicos que “funcionam bem”:

- 0, 64, 128, 192 (degraus)

Ou use NVIC_SetPriority() e deixe o CMSIS ajustar.

---

6) “Receitas” prontas (macros auxiliares didáticas)

Essas macros são úteis para o leitor entender o que acontece. Em projetos reais, pode ser mais conveniente usar `NVIC_EnableIRQ()`, `NVIC_SetPriority()` e funções equivalentes do CMSIS. Aqui, a manipulação direta dos registradores foi mantida por valor didático.

```
#define NVIC_IRQ_BIT(irqn)   (1u << (uint32_t)(irqn))


static inline void nvic_enable_irq(uint32_t irqn)
{
   NVIC->ISER[0] = NVIC_IRQ_BIT(irqn);
}


static inline void nvic_disable_irq(uint32_t irqn)
{
   NVIC->ICER[0] = NVIC_IRQ_BIT(irqn);
```

```
}

static inline void nvic_set_pending(uint32_t irqn)
{
    NVIC->ISPR[0] = NVIC_IRQ_BIT(irqn);
}

static inline void nvic_clear_pending(uint32_t irqn)
{
    NVIC->ICPR[0] = NVIC_IRQ_BIT(irqn);
}
```

Para prioridade, como IP[] é “word array”, o CMSIS normalmente tem funções prontas. Mas dá para fazer manualmente também:

```
static inline void nvic_set_priority(uint32_t irqn, uint8_t prio)
{
    /* Cada IRQ usa 8 bits: IPR byte index = irqn */
    volatile uint8_t *ip = (volatile uint8_t *)NVIC->IP;
    ip[irqn] = prio;
}

static inline uint8_t nvic_get_priority(uint32_t irqn)
{
    volatile uint8_t *ip = (volatile uint8_t *)NVIC->IP;
    return ip[irqn];
}
```

---

7) Exemplos replicáveis no STM32CubeIDE

Vamos usar uma IRQ externa “qualquer” definida no device header. No STM32G0, uma boa escolha é USART1_IRQn (como fizemos antes), porque existe e o handler é conhecido.

✅ **Só vamos habilitar/pendar a IRQ via NVIC para ver o handler rodar.**

7.1 Arquivo nvic_examples.h

```
#ifndef NVIC_EXAMPLES_H

#define NVIC_EXAMPLES_H


#include <stdint.h>


void nvic_examples_run(void);


/* Observáveis */

extern volatile uint32_t g_irq_hits;

extern volatile uint32_t g_case_step;


extern volatile uint32_t g_iser;

extern volatile uint32_t g_icer;

extern volatile uint32_t g_ispr;

extern volatile uint32_t g_icpr;


extern volatile uint8_t  g_prio_before;

extern volatile uint8_t  g_prio_after;


#endif
```

7.2 Arquivo nvic_examples.c

```
#include "nvic_examples.h"

#include "stm32g0xx.h"
```

```
#include "core_cm0plus.h"

/* Observáveis no debugger */
volatile uint32_t g_irq_hits = 0;
volatile uint32_t g_case_step = 0;

volatile uint32_t g_iser = 0;
volatile uint32_t g_icer = 0;
volatile uint32_t g_ispr = 0;
volatile uint32_t g_icpr = 0;

volatile uint8_t g_prio_before = 0;
volatile uint8_t g_prio_after  = 0;

#define NVIC_IRQ_BIT(irqn) (1u << (uint32_t)(irqn))

static inline void nvic_enable_irq(uint32_t irqn)   { NVIC->ISER[0] =
NVIC_IRQ_BIT(irqn); }
static inline void nvic_disable_irq(uint32_t irqn)  { NVIC->ICER[0] =
NVIC_IRQ_BIT(irqn); }
static inline void nvic_set_pending(uint32_t irqn)  { NVIC->ISPR[0] =
NVIC_IRQ_BIT(irqn); }
static inline void nvic_clear_pending(uint32_t irqn){ NVIC->ICPR[0] =
NVIC_IRQ_BIT(irqn); }

static inline void nvic_set_priority(uint32_t irqn, uint8_t prio)
{
    volatile uint8_t *ip = (volatile uint8_t *)NVIC->IP;
    ip[irqn] = prio;
```

```
}

static inline uint8_t nvic_get_priority(uint32_t irqn)
{
    volatile uint8_t *ip = (volatile uint8_t *)NVIC->IP;
    return ip[irqn];
}

/* Handler da IRQ escolhida (não mexe em periférico ST) */
void USART1_IRQHandler(void)
{
    g_irq_hits++;
}

/* Snapshot dos regs do NVIC (somente para observação) */
static void snap_nvic_regs(void)
{
    g_iser = NVIC->ISER[0];
    g_icer = NVIC->ICER[0];
    g_ispr = NVIC->ISPR[0];
    g_icpr = NVIC->ICPR[0];
}

void nvic_examples_run(void)
{
    const uint32_t irqn = (uint32_t)USART1_IRQn;

    /* Caso 1: desabilita e tenta pendurar -> não deve entrar (fica pending) */
```

```
g_case_step = 1;
g_irq_hits = 0;

nvic_disable_irq(irqn);
nvic_clear_pending(irqn);
snap_nvic_regs();
__NOP(); /* breakpoint A: veja ISER/ICER/ISPR/ICPR e g_irq_hits=0 */

nvic_set_pending(irqn);
snap_nvic_regs();
__NOP(); /* breakpoint B: IRQ fica pending, mas não executa porque está disabled */

/* Caso 2: habilita -> se estava pending, deve executar imediatamente */
g_case_step = 2;
nvic_enable_irq(irqn);
__DMB();
__NOP(); /* breakpoint C: após enable, g_irq_hits deve ter incrementado */

/* Caso 3: prioridade (quantização no M0+) */
g_case_step = 3;
g_prio_before = nvic_get_priority(irqn);

nvic_set_priority(irqn, 3); /* tentativa: prioridade baixa */
g_prio_after = nvic_get_priority(irqn);

__NOP(); /* breakpoint D: compare prio_before e prio_after (quantizado) */

/* Caso 4: limpar pending (boa prática antes de habilitar) */
```

```
  g_case_step = 4;
  nvic_set_pending(irqn);
  nvic_clear_pending(irqn);
  snap_nvic_regs();

  __NOP(); /* breakpoint E */
}
```

7.3 Chamada no main.c

```
#include "nvic_examples.h"

int main(void)
{
 HAL_Init();
 SystemClock_Config();

 nvic_examples_run();

 while (1) { }
}
```

Como o leitor valida (passo a passo)

Breakpoint A

- IRQ desabilitada
- g_irq_hits = 0
- g_iser deve mostrar bit do USART1 0
- g_icer não é “estado”, é write-1-to-clear (snapshot pode não ser muito informativo). O estado real é lido no ISER.

Breakpoint B

- Set pending com IRQ desabilitada:

o g_ispr pode mostrar pendência (dependendo de como o snapshot lê o registro)

o ISR não executa (g_irq_hits continua 0)

Breakpoint C

- Habilita IRQ:

o Se estava pending, o NVIC chama o handler e g_irq_hits sobe.

Breakpoint D

- Prioridade:

o g_prio_after pode não ser “3” literal; pode virar um valor quantizado (ex.: 192 etc.)

o Isso prova que só bits altos são implementados no M0+.

Breakpoint E

- Clear pending:

o Mostra a receita de limpar pendência antes de reabilitar ou ao “resetar” fluxo.

Erros clássicos de NVIC que esse exemplo evita

1. Habilitar IRQ sem limpar pending → ISR dispara “do nada”

2. Achar que prioridade tem 256 níveis no M0+ → na prática são poucos níveis

3. Escrever 0 em ISER/ICER achando que desabilita/habilita → é write-1-to-set/clear

---

1) Reset_Handler: quando é chamado e o que ele faz

1.1 Quando é chamado

O Reset_Handler é a primeira função do seu firmware executada após um reset (power-on reset, reset por NRST, watchdog, reset por software etc.).

O core:

1. lê o primeiro word da vector table e carrega no MSP (stack pointer inicial)

2. lê o segundo word e salta para o Reset_Handler

1.2 O que o Reset_Handler faz (sequência típica “startup” STM32)

Em um projeto STM32CubeIDE (GCC), o Reset_Handler normalmente executa esta ordem:

1. Configura stack (na prática já veio do vetor [0])
2. Copia .data da Flash → RAM

o variáveis globais inicializadas (ex.: int x = 5;) precisam ser copiadas

3. Zera .bss na RAM

o variáveis globais não inicializadas (ex.: int y;) começam em 0

4. Chama SystemInit() (configurações iniciais do core/clock)
5. Chama __libc_init_array() (construtores C++ e init de runtime, quando aplicável)
6. Chama main()

Essa lógica é exatamente o que garante o “comportamento mágico”:

- global inicializada tem valor correto logo no começo do main
- global não inicializada começa em 0

1.3 Exemplo REAL e testável: provar .data e .bss (sem periféricos)

Crie um arquivo reset_proof.c em Core/Src:

```
#include <stdint.h>

/* Vai para .data (flash -> ram) */
uint32_t g_data_init = 0x12345678u;

/* Vai para .bss (ram, zerado no Reset_Handler) */
uint32_t g_bss_zero;

/* Variáveis para observar no debugger */
volatile uint32_t g_seen_data;
volatile uint32_t g_seen_bss;
```

```
void reset_proof_run(void)
{
   /* Coloque breakpoint aqui no começo do main e inspecione:
      g_data_init deve ser 0x12345678
      g_bss_zero deve ser 0x00000000 */
   g_seen_data = g_data_init;
   g_seen_bss  = g_bss_zero;
}
```

No main.c, chame logo no início (depois do clock, ou até antes, tanto faz para isso):

```
extern void reset_proof_run(void);

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  reset_proof_run();

  while (1) { }
}
```

✅ **Como o leitor valida**

- Breakpoint dentro de reset_proof_run()
- Watch:

o g_data_init == 0x12345678

o g_bss_zero == 0

Se isso falhar, o Reset_Handler/startup/linker script estão errados — é um teste ótimo.

---

2) Tabela de vetores: como montar corretamente

2.1 Estrutura da vector table (Cortex-M)

A tabela é um array de endereços (palavras de 32 bits) em memória:

Índice	Conteúdo
0	MSP inicial
1	Reset_Handler
2	NMI_Handler
3	HardFault_Handler
…	…
14	PendSV_Handler
15	SysTick_Handler
16	IRQ0 (primeira IRQ externa)
17	IRQ1
…	…

O datasheet descreve a ideia: o endereço do handler é construído a partir de um base address (vector table base) + offset pelo número do vetor.

O mecanismo detalhado é do ARMv6-M/PM0223.

✅ **Fórmula prática para IRQ externa:**

slot = 16 + IRQn

Onde IRQn é o valor do enum IRQn_Type no CMSIS (por exemplo, USART1_IRQn).

2.2 "Montar olhando a tabela de interrupções" (documentação) + validação no código

O processo correto no STM32 é:

1. Pegue a lista (documentação do chip) ou o IRQn_Type do CMSIS
2. Para cada interrupção, determine o IRQn (número)
3. A entrada na vector table é vector[16 + IRQn] = endereço_do_handler

Dica: Use a tabela do manual/datasheet para saber quais IRQs existem, mas para não errar a numeração, confira sempre o IRQn_Type no header (isso bate com o startup e com o NVIC).

---

3) Exemplo REAL e testável: "ler a vector table atual" e provar o slot de uma IRQ

Esse exemplo é excelente porque:

- não precisa mexer no startup
- não precisa periférico ST
- prova a relação: IRQn → slot

Crie vector_table_probe.c:

```
#include "stm32g0xx.h"      // ou stm32g070xx.h, conforme seu projeto
#include "core_cm0plus.h"
#include <stdint.h>

/* Observáveis no debugger */
volatile uint32_t g_vtor_addr;
volatile uint32_t g_vec_reset;
volatile uint32_t g_vec_systick;
volatile uint32_t g_vec_usart1_slot;
volatile uint32_t g_expected_usart1;

/* Declara o handler (o nome precisa existir no seu projeto) */
void USART1_IRQHandler(void);

void vector_table_probe_run(void)
{
   /* Endereço base atual da tabela de vetores */
   uint32_t *vt = (uint32_t *)SCB->VTOR;
   g_vtor_addr = (uint32_t)vt;
```

```
  /* Entradas padrão (system exceptions) */
  g_vec_reset   = vt[1];   // Reset_Handler
  g_vec_systick = vt[15];  // SysTick_Handler

  /* IRQ externa: slot = 16 + IRQn */
  uint32_t slot = 16u + (uint32_t)USART1_IRQn;
  g_vec_usart1_slot = vt[slot];

  /* Endereço esperado do handler (Thumb => bit0=1) */
  g_expected_usart1 = ((uint32_t)USART1_IRQHandler) | 1u;

  __NOP(); /* breakpoint: compare g_vec_usart1_slot com g_expected_usart1 */
}
```

No main.c, chame:

```
extern void vector_table_probe_run(void);

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  vector_table_probe_run();

  while (1) { }
}
```

✅ **Como o leitor valida**

- Breakpoint no __NOP()

- Veja no Watch:
  - g_vtor_addr (onde a tabela está)
  - g_vec_usart1_slot (valor dentro da tabela)
  - g_expected_usart1 (endereço do handler)
- Eles devem bater.

Esse teste prova, com o hardware rodando, que a montagem 16 + IRQn está correta.

---

4) Exemplo REAL e testável: montar uma vector table "mínima" em RAM e trocar VTOR

Agora vamos fazer algo mais avançado e muito didático:

- copiar a tabela para RAM
- trocar uma entrada
- apontar VTOR para RAM
- disparar a IRQ por software (NVIC pending)
- ver que o handler alternativo executa

Crie vector_table_relocate_demo.c:

```
#include "stm32g0xx.h"

#include "core_cm0plus.h"

#include <stdint.h>


#define VECTOR_COUNT  48u  /* suficiente para system + várias IRQs */


static uint32_t g_vt_ram[VECTOR_COUNT] __attribute__((aligned(256)));


volatile uint32_t g_alt_hits = 0;


/* handler alternativo */

static void alt_usart1_handler(void)
```

```
{
    g_alt_hits++;
}

/* handler “normal” (só para existir) */
void USART1_IRQHandler(void)
{
    /* não mexe em periférico ST */
}

/* demo */
void vector_table_relocate_demo_run(void)
{
    uint32_t *src = (uint32_t *)SCB->VTOR;

    /* 1) copia tabela atual */
    for (uint32_t i = 0; i < VECTOR_COUNT; i++)
        g_vt_ram[i] = src[i];

    /* 2) troca USART1 slot */
    uint32_t slot = 16u + (uint32_t)USART1_IRQn;
    g_vt_ram[slot] = ((uint32_t)alt_usart1_handler) | 1u;

    /* 3) muda VTOR */
    __DMB();
    SCB->VTOR = (uint32_t)g_vt_ram;
    __DMB();
    __ISB();
```

```
  /* 4) habilita + pendura USART1 para testar */
  NVIC_ClearPendingIRQ(USART1_IRQn);
  NVIC_EnableIRQ(USART1_IRQn);
  NVIC_SetPendingIRQ(USART1_IRQn);

  __NOP(); /* breakpoint: g_alt_hits deve ser >= 1 */
}
```

Chame no main e valide:

- g_alt_hits incrementou
- SCB->VTOR aponta para g_vt_ram

---

5) “Montar a tabela olhando a tabela de interrupções do chip”

Mesmo sem eu listar aqui todas as IRQs (porque isso vira uma tabela gigante do manual), o procedimento didático é:

1. Abra a tabela “Interrupt and exception vectors” do manual do chip
2. Para cada item, anote:

o nome da interrupção (ex.: USART1)

o o IRQn correspondente (o CMSIS já define o USART1_IRQn)

3. A entrada do vetor fica em:

o vector[16 + USART1_IRQn] = USART1_IRQHandler

✅ **Regra de ouro para não errar**

- Sempre confirme no IRQn_Type do CMSIS do seu projeto. É ele que o startup usa.

A seguir está um HardFault_Handler “debugger-friendly” para Cortex-M0+ (STM32G070RB) que captura o stack frame empilhado pelo hardware e deixa tudo em variáveis globais fáceis de inspecionar no Watch do STM32CubeIDE.

No Cortex-M0+, quase todo fault vira HardFault, então esse handler é uma das ferramentas mais valiosas do livro. O mecanismo de stacking automático (r0–r3, r12, lr, pc, xPSR) é definido pela arquitetura ARMv6-M.

---

1) O que o core empilha automaticamente ao entrar em HardFault

Quando ocorre uma exceção (incluindo HardFault), o Cortex-M empilha automaticamente no stack atual:

1. r0
2. r1
3. r2
4. r3
5. r12
6. lr (return address / EXC_RETURN)
7. pc (endereço onde estava executando)
8. xPSR

Isso é o stack frame que vamos “dump” para debug.

---

2) Código completo e testável

2.1 hardfault_dump.h (Core/Inc)

```
#ifndef HARDFAULT_DUMP_H

#define HARDFAULT_DUMP_H


#include <stdint.h>


/* Estrutura com o frame empilhado automaticamente */

typedef struct

{
```

```
    uint32_t r0;
    uint32_t r1;
    uint32_t r2;
    uint32_t r3;
    uint32_t r12;
    uint32_t lr;
    uint32_t pc;
    uint32_t xpsr;
} hf_stack_frame_t;

/* Variáveis globais para observar no debugger */
extern volatile hf_stack_frame_t g_hf_frame;
extern volatile uint32_t g_hf_msp;
extern volatile uint32_t g_hf_psp;
extern volatile uint32_t g_hf_primask;
extern volatile uint32_t g_hf_control;
extern volatile uint32_t g_hf_ipsr;
extern volatile uint32_t g_hf_vtor;

/* Função auxiliar (opcional) para forçar um fault de teste */
void hardfault_force_test(void);

#endif /* HARDFAULT_DUMP_H */
```

2.2 hardfault_dump.c (Core/Src)

```
#include "hardfault_dump.h"
#include "stm32g0xx.h"
#include "core_cm0plus.h"
```

```
/* Globais observáveis */
volatile hf_stack_frame_t g_hf_frame;
volatile uint32_t g_hf_msp     = 0;
volatile uint32_t g_hf_psp     = 0;
volatile uint32_t g_hf_primask = 0;
volatile uint32_t g_hf_control = 0;
volatile uint32_t g_hf_ipsr    = 0;
volatile uint32_t g_hf_vtor    = 0;

/* Função chamada pelo handler para copiar o frame */
static void hf_capture(uint32_t *stack_ptr)
{
   /* Captura contexto do core */
   g_hf_msp     = __get_MSP();
   g_hf_psp     = __get_PSP();
   g_hf_primask = __get_PRIMASK();
   g_hf_control = __get_CONTROL();
   g_hf_ipsr    = __get_IPSR();
   g_hf_vtor    = SCB->VTOR;

   /* Copia o stack frame empilhado automaticamente */
   g_hf_frame.r0   = stack_ptr[0];
   g_hf_frame.r1   = stack_ptr[1];
   g_hf_frame.r2   = stack_ptr[2];
   g_hf_frame.r3   = stack_ptr[3];
   g_hf_frame.r12  = stack_ptr[4];
   g_hf_frame.lr   = stack_ptr[5];
```

```
    g_hf_frame.pc   = stack_ptr[6];
    g_hf_frame.xpsr = stack_ptr[7];
}

/* Handler “C” (não é chamado diretamente pela vector table) */
void HardFault_Handler_C(uint32_t *stack_ptr)
{
    hf_capture(stack_ptr);

    /* Pare aqui para inspecionar no debugger */
    __BKPT(0);

    /* Trava em loop: em produto, você pode resetar ou ir para safe state */
    while (1)
    {
        __NOP();
    }
}

/* HardFault_Handler real (chamado pela vector table)
 *
 * No Cortex-M0+, normalmente estamos usando MSP.
 * Mesmo assim, usamos o valor atual do SP para pegar o frame.
 */
__attribute__((naked)) void HardFault_Handler(void)
{
    __asm volatile
    (
```

```
        "mov r0, sp       \n" /* r0 = stack pointer atual */
        "b   HardFault_Handler_C \n"
    );
}

/* Forçar HardFault para teste (opcional)
 * Causa um salto para endereço inválido.
 */
void hardfault_force_test(void)
{
    /* Ponteiro de função para endereço inválido (Thumb bit set para tentar executar) */
    void (*bad_fn)(void) = (void (*)(void))0x00000001u;
    bad_fn(); /* deve gerar HardFault */
}
```

---

3) Como testar no STM32CubeIDE (passo a passo)

Passo 1 — Adicionar ao projeto

- Coloque hardfault_dump.h em Core/Inc
- Coloque hardfault_dump.c em Core/Src

Passo 2 — Chamar o teste (opcional)

No main.c, depois do clock:

```
#include "hardfault_dump.h"

int main(void)
{
  HAL_Init();
  SystemClock_Config();
```

```
  /* ATENÇÃO: isso vai causar HardFault propositalmente */
  // hardfault_force_test();


  while (1) { }
}
```

✅ **Para testar, descomente hardfault_force_test();, rode em Debug.**

Passo 3 — Quando travar, inspecionar variáveis

Quando o HardFault ocorrer, o código entra em __BKPT(0) e para no debugger.

Abra o Watch e inspecione:

- g_hf_frame.pc → onde estava executando quando faultou
- g_hf_frame.lr → retorno
- g_hf_frame.xpsr → flags + IPSR
- g_hf_msp e g_hf_psp → stacks
- g_hf_vtor → base da vector table

✅ **Agora você consegue ir no Disassembly/Source e ver exatamente a instrução no PC que causou o problema.**

---

4) Como interpretar os valores (guia rápido)

4.1 pc é o ouro

- Se pc aponta para um endereço "maluco" (ex.: 0xFFFFFFF…), suspeite de stack corrompida / ponteiro inválido.
- Se pc aponta para uma função sua, abra no disassembly e veja a linha.

4.2 lr ajuda a entender o caminho

- lr pode apontar para onde a função retornaria
- ou conter EXC_RETURN em alguns cenários de exceção

4.3 xpsr e ipsr

- g_hf_ipsr vai indicar que você está no HardFault handler (não será 0).
- xpsr também contém o IPSR embutido.

---

5) Erros típicos que esse dump resolve rapidamente

- buffer overflow (corrompe retorno → PC vira lixo)
- ponteiro de função inválido
- acesso a endereço errado (registrador/periférico inválido)
- stack overflow (stack invade outras áreas)
- ISR com stack frame grande demais

---

6) Versão "produção"

Em produto, em vez de BKPT + loop infinito, você pode:

- gravar g_hf_frame em uma área "noinit" da RAM (para persistir após reset)
- solicitar reset via SCB->AIRCR (SYSRESETREQ)
- entrar em safe state

Nos blocos seguintes, esse mecanismo evolui para um dump persistente em `.noinit`, ainda sem depender de periféricos da ST.

A seguir vamos fechar o "core-side" com NMI, SVC e PendSV no Cortex-M0+ (STM32G070RB), bem detalhado e com exemplos testáveis no STM32CubeIDE. Vamos validar tudo com breakpoints + Watch + Disassembly.

---

1) NMI_Handler (Non-Maskable Interrupt)

1.1 O que é NMI

NMI é uma exceção especial do Cortex-M:

- tem prioridade muito alta (acima de IRQs normais)
- não pode ser mascarada por PRIMASK (__disable_irq() não impede NMI)
- é usada para eventos críticos (em muitos MCUs: falhas graves de clock, watchdog crítico, etc. — as fontes em si são do fabricante, mas a exceção NMI é do core)

✅ **Regra prática:**

Se chegou em NMI, algo "grave" aconteceu ou você forçou por software.

1.2 Quando NMI é chamada (core)

- Quando o core recebe a exceção NMI.
- Você também pode forçar NMI por software: SCB->ICSR.NMIPENDSET = 1.

⚠ **Isso é ótimo para teste didático, mas em produto NMI deve ser minimalista e robusta.**

1.3 Como escrever um NMI "profissional"

Boas práticas:

- fazer o mínimo possível
- registrar o evento (ex.: contador, snapshot de registradores)
- evitar loops longos, printf, alocação, etc.
- se for falha de clock, encaminhar para safe state ou reset (isso entra em periféricos ST depois)

1.4 Exemplo testável: forçar NMI por software (SCB->ICSR)

nmi_demo.c (Core/Src)

```
#include "stm32g0xx.h"

#include "core_cm0plus.h"

#include <stdint.h>


volatile uint32_t g_nmi_hits = 0;

volatile uint32_t g_nmi_ipsr = 0;


void NMI_Handler(void)

{

  g_nmi_hits++;

  g_nmi_ipsr = __get_IPSR();   /* deve indicar exceção NMI */

  __BKPT(0);                   /* para o debugger parar aqui */

}
```

```
void nmi_force_demo(void)
{
   g_nmi_hits = 0;
   g_nmi_ipsr = 0;

   __NOP(); /* breakpoint A: antes de pendurar NMI */

   /* Força NMI: NMIPENDSET (bit 31 do ICSR) */
   SCB->ICSR = (1u << 31);

   __NOP(); /* breakpoint B: só chega aqui depois de sair do NMI_Handler */
}
```

Como testar

1. Chame nmi_force_demo() no main() (após clock).
2. Rode em Debug.
3. Vai parar no BKPT dentro do NMI_Handler.
4. Observe:

o g_nmi_hits == 1

o g_nmi_ipsr ≠ 0 (indica exceção ativa)

5. Dê “Resume” e veja que volta para o fluxo.

✅ **Prova essencial:**

- Mesmo se você inserir __disable_irq() antes de SCB->ICSR = ..., o NMI ainda acontece.

---

2) SVC (Supervisor Call) — SVC_Handler

2.1 O que é SVC

SVC é uma exceção usada para solicitar um “serviço” ao sistema operacional ou ao firmware, tipicamente:

- trocar privilégio (em sistemas com thread unprivileged)
- implementar “syscalls” em RTOS
- criar uma “porta” controlada para operações sensíveis

Em bare-metal, você pode usar SVC para:

- criar um dispatcher de serviços
- chamar rotinas críticas de forma padronizada

2.2 Como SVC é disparado

Por uma instrução assembly:

- SVC #imm8 (um número de 0 a 255 embutido na instrução)

Esse número (imm8) costuma ser usado como ID do serviço.

✅ **No C, você pode disparar SVC via inline asm ou via wrapper.**

2.3 Como pegar o número do SVC (a parte “mágica”)

Quando a exceção acontece, o core empilha o stack frame, incluindo o PC.

O PC empilhado aponta para a próxima instrução após o SVC.

A instrução SVC está logo antes do PC empilhado, então:

- endereço da instrução SVC = pc_stacked - 2 (Thumb, 16-bit)

O byte imediato do SVC está no opcode. Para Thumb SVC, normalmente:

- opcode = 0xDFxx, onde xx é o imediato.

2.4 Exemplo testável: mini “syscall” via SVC

svc_demo.c (Core/Src)

```
#include "stm32g0xx.h"

#include "core_cm0plus.h"

#include <stdint.h>


volatile uint32_t g_svc_hits = 0;
```

```
volatile uint32_t g_svc_num  = 0;
volatile uint32_t g_svc_r0_in = 0;
volatile uint32_t g_svc_r0_out = 0;

static void SVC_Handler_C(uint32_t *stack_ptr)
{
   /* stack_ptr aponta para: r0,r1,r2,r3,r12,lr,pc,xpsr */
   uint32_t pc = stack_ptr[6];

   /* instrução SVC está em pc-2 (Thumb 16-bit) */
   uint16_t *svc_insn = (uint16_t *)(pc - 2u);
   uint16_t op = *svc_insn;

   /* opcode SVC em Thumb: 0xDFxx */
   uint8_t imm = (uint8_t)(op & 0x00FFu);

   g_svc_hits++;
   g_svc_num  = imm;
   g_svc_r0_in = stack_ptr[0];

   /* Exemplo de “serviços”:
     SVC 0x01: retorna r0+1
     SVC 0x02: retorna r0*2
   */
   if (imm == 0x01u)
      stack_ptr[0] = stack_ptr[0] + 1u;
   else if (imm == 0x02u)
      stack_ptr[0] = stack_ptr[0] * 2u;
```

```
    g_svc_r0_out = stack_ptr[0];

    __BKPT(0);
}

/* Handler real — naked para pegar SP e chamar C */
__attribute__((naked)) void SVC_Handler(void)
{
    __asm volatile(
        "mov r0, sp \n"
        "b   SVC_Handler_C \n"
    );
}

/* Wrappers para chamar SVC com imediato fixo */
static inline uint32_t svc_call_01(uint32_t x)
{
    register uint32_t r0 __asm("r0") = x;
    __asm volatile ("svc 0x01" : "+r"(r0) :: "memory");
    return r0;
}

static inline uint32_t svc_call_02(uint32_t x)
{
    register uint32_t r0 __asm("r0") = x;
    __asm volatile ("svc 0x02" : "+r"(r0) :: "memory");
    return r0;
```

```
}

void svc_demo_run(void)
{
   g_svc_hits = 0;
   g_svc_num  = 0;

   __NOP(); /* breakpoint A */

   uint32_t a = svc_call_01(10); /* espera 11 */
   __NOP(); /* breakpoint B: a deve ser 11 */

   uint32_t b = svc_call_02(10); /* espera 20 */
   __NOP(); /* breakpoint C: b deve ser 20 */

   (void)a;
   (void)b;
}
```

Como testar

1. Chame svc_demo_run() no main().
2. Rode em Debug.
3. Vai parar no BKPT dentro do SVC handler em cada chamada.
4. Observe:

- o g_svc_num (0x01 ou 0x02)
- o g_svc_r0_in e g_svc_r0_out
- o variável local a e b no breakpoint B/C

✅ **Isso mostra:**

- como o número do serviço é recuperado do opcode do SVC

- como retornar valor ao chamador alterando r0 no stack frame

---

3) PendSV (Pendable Service Call) — PendSV_Handler

3.1 O que é PendSV

PendSV é uma exceção “pendável” projetada para:

- ser disparada por software
- ser usada como “trabalho deferido”
- (principal uso) context switch em RTOS

A diferença dele para SVC:

- SVC é “chamada imediata” (trap)
- PendSV é “agendada” (pendente), geralmente executada quando o sistema está pronto

---

3.2 Como PendSV é disparado

Via SCB->ICSR.PENDSVSET = 1 (bit 28).

E pode ser limpo com PENDSVCLR (bit 27).

3.3 Exemplo testável: set/clear PendSV e observar VECTACTIVE

pendsv_demo.c (Core/Src)

```
#include "stm32g0xx.h"

#include "core_cm0plus.h"

#include <stdint.h>


volatile uint32_t g_pendsv_hits = 0;

volatile uint32_t g_pendsv_ipsr = 0;


void PendSV_Handler(void)

{
```

```
    g_pendsv_hits++;
    g_pendsv_ipsr = __get_IPSR(); /* indica exceção PendSV */
    __BKPT(0);
}

void pendsv_demo_run(void)
{
    g_pendsv_hits = 0;
    g_pendsv_ipsr = 0;

    __NOP(); /* breakpoint A */

    /* Limpa pendência e seta pendência */
    SCB->ICSR = (1u << 27); /* PENDSVCLR */
    SCB->ICSR = (1u << 28); /* PENDSVSET */

    __NOP(); /* breakpoint B: após sair do PendSV_Handler */
}
```

Como testar

1. Chame pendsv_demo_run() no main().
2. Rode em Debug.
3. Para no BKPT dentro do PendSV_Handler.
4. Veja:

- o g_pendsv_hits == 1
- o g_pendsv_ipsr indicando exceção ativa (não 0)

---

4) Amarrando os três (ordem e prioridade)

Agora que você tem:

- NMI (não mascarável)
- SVC (trap)
- PendSV (pendável)

Você pode fazer um teste muito interessante (só no core):

- pendurar PendSV
- chamar SVC
- forçar NMI

E ver a ordem em que eles interrompem uns aos outros (dependendo do momento).

✅ **Regra geral:**

- NMI tem prioridade altíssima, entra "por cima" de quase tudo.
- HardFault também é altíssimo.
- SysTick/SVC/PendSV são configuráveis (SHP), mas PendSV normalmente fica com prioridade mais baixa em RTOS.

---

5) Runner único

Se quiser, você pode chamar na sequência no main():

```
extern void nmi_force_demo(void);
extern void svc_demo_run(void);
extern void pendsv_demo_run(void);

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  /* teste 1: SVC */
  svc_demo_run();
```

```
  /* teste 2: PendSV */
  pendsv_demo_run();

  /* teste 3: NMI (por último, porque é “mais agressivo”) */
  nmi_force_demo();

  while (1) { }
}
```

A seguir vamos ajustar prioridades de SVC, PendSV e SysTick (system handlers) no Cortex-M0+, e montar um teste reprodutível para o leitor enxergar a ordem de execução no debugger.

Lembrete importante do M0+: a prioridade é quantizada (poucos bits efetivos). Você vai ver valores “arredondados”. Isso é esperado.

---

1) Onde ficam as prioridades de SVC/PendSV/SysTick

Essas exceções são system handlers, então não usam NVIC->IP[] como IRQ externa.

Elas usam os registradores SCB->SHP[] (System Handler Priority) — mas o CMSIS já fornece um caminho simples:

✅ **Use NVIC_SetPriority() mesmo para SysTick_IRQn, PendSV_IRQn e SVCall_IRQn.**

O CMSIS decide se escreve em NVIC->IP (IRQ externa) ou SCB->SHP (system).

Exemplos:

```
NVIC_SetPriority(SysTick_IRQn,  3);
NVIC_SetPriority(PendSV_IRQn,   3);
NVIC_SetPriority(SVCall_IRQn,   0);
```

---

2) Objetivo do teste (o que o leitor vai ver)

Vamos criar um cenário em que:

- SysTick está rodando (gera ticks)

- PendSV é pendurado
- SVC é chamado imediatamente

E queremos observar:

1. SVC acontece “na hora” (trap)
2. PendSV só roda depois, por estar com prioridade mais baixa (RTOS-style)
3. SysTick pode ocorrer entre eles dependendo do timing e prioridades

Para ficar “visível”, vamos gravar uma trilha (trace) em um array global.

---

3) Código testável (STM32CubeIDE)

3.1 Arquivo prio_demo.h

```
#ifndef PRIO_DEMO_H
#define PRIO_DEMO_H

#include <stdint.h>

void prio_demo_run(void);

/* Observáveis */
extern volatile uint8_t  g_prio_trace[64];
extern volatile uint32_t g_prio_trace_len;

extern volatile uint32_t g_prio_systick;
extern volatile uint32_t g_prio_pendsv;
extern volatile uint32_t g_prio_svcall;

extern volatile uint32_t g_hits_systick;
extern volatile uint32_t g_hits_pendsv;
extern volatile uint32_t g_hits_svc;
```

```
#endif
```

3.2 Arquivo prio_demo.c

```
#include "prio_demo.h"
#include "stm32g0xx.h"
#include "core_cm0plus.h"

/* Trace */
volatile uint8_t  g_prio_trace[64];
volatile uint32_t g_prio_trace_len = 0;

/* Prioridades lidas */
volatile uint32_t g_prio_systick = 0;
volatile uint32_t g_prio_pendsv  = 0;
volatile uint32_t g_prio_svcall  = 0;

/* Contadores */
volatile uint32_t g_hits_systick = 0;
volatile uint32_t g_hits_pendsv  = 0;
volatile uint32_t g_hits_svc     = 0;

static void trace_push(uint8_t code)
{
   if (g_prio_trace_len < sizeof(g_prio_trace))
      g_prio_trace[g_prio_trace_len++] = code;
}

/* -------------------------
```

```
 * SysTick config (1ms)
 * ------------------------- */
static void systick_config_1ms(void)
{
    uint32_t ticks = (SystemCoreClock / 1000u) - 1u;

    SysTick->CTRL = 0;
    SysTick->LOAD = ticks;
    SysTick->VAL  = 0;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk   |
                    SysTick_CTRL_ENABLE_Msk;
}

/* -------------------------
 * Handlers
 * ------------------------- */
void SysTick_Handler(void)
{
    g_hits_systick++;
    trace_push('T'); /* Tick */
}

void PendSV_Handler(void)
{
    g_hits_pendsv++;
    trace_push('P'); /* PendSV */
}
```

```
static void SVC_Handler_C(uint32_t *sp)
{
    (void)sp;
    g_hits_svc++;
    trace_push('S'); /* SVC */
}

/* naked para pegar SP e chamar C */
__attribute__((naked)) void SVC_Handler(void)
{
    __asm volatile(
        "mov r0, sp \n"
        "b   SVC_Handler_C \n"
    );
}

/* Wrapper para gerar SVC (serviço 0x01 só para existir) */
static inline void svc_call(void)
{
    __asm volatile("svc 0x01" ::: "memory");
}

/* -------------------------
 * Demo
 * ------------------------- */
void prio_demo_run(void)
{
```

```
/* Zera observáveis */
g_prio_trace_len = 0;
g_hits_systick = 0;
g_hits_pendsv  = 0;
g_hits_svc     = 0;

/* 1) Definir prioridades (RTOS-style)
 * - PendSV: mais baixa (deferido)
 * - SysTick: baixa/média (tick)
 * - SVC: mais alta entre as três (trap imediato)
 *
 * No M0+ só poucos bits valem => valores podem “quantizar”.
 */
NVIC_SetPriority(PendSV_IRQn,  3);
NVIC_SetPriority(SysTick_IRQn, 2);
NVIC_SetPriority(SVCall_IRQn,  0);

g_prio_pendsv  = NVIC_GetPriority(PendSV_IRQn);
g_prio_systick = NVIC_GetPriority(SysTick_IRQn);
g_prio_svcall  = NVIC_GetPriority(SVCall_IRQn);

/* 2) Start SysTick */
systick_config_1ms();

__NOP(); /* breakpoint A: veja prioridades e counters */

/* 3) Pendurar PendSV (não executa imediatamente se houver trabalho mais
prioritário) */
```

```
  SCB->ICSR = (1u << 28); /* PENDSVSET */
  trace_push('p');       /* marca “pendurei PendSV” */

  /* 4) Chamar SVC (trap imediato) */
  trace_push('c');       /* marca “vou chamar SVC” */
  svc_call();
  trace_push('r');       /* marca “retornei do SVC” */

  /* 5) Aguarde um pouco em busy wait para deixar SysTick e PendSV acontecerem */
  for (volatile uint32_t i = 0; i < 200000u; i++)
  {
    __NOP();
  }

  __NOP(); /* breakpoint B: inspecione g_prio_trace */
}
```

---

4) Como usar no main.c

```
#include "prio_demo.h"

int main(void)
{
 HAL_Init();
 SystemClock_Config();

 prio_demo_run();

 while (1) { }
```

}

---

5) Como você interpreta o resultado (debugger)

Breakpoint A

Observe:

- g_prio_svcall, g_prio_systick, g_prio_pendsv

o podem aparecer como 0/128/192 etc. (quantização do M0+)

- counters ainda 0 ou começando a subir (SysTick pode ter começado)

Breakpoint B

Abra o Watch e veja:

- g_hits_svc deve ser 1
- g_hits_pendsv deve ser ≥ 1 (se o PendSV rodou)
- g_hits_systick deve ser > 0

Veja o trace em g_prio_trace[] (primeiros bytes):

- 'p' = pendurei PendSV
- 'c' = vou chamar SVC
- 'S' = SVC handler executou
- 'r' = voltei do SVC
- 'P' = PendSV executou (deveria vir depois)
- 'T' = SysTick tick (pode aparecer em vários lugares)

O que você espera ver (típico)

Algo como:

p c S r ... P ... T T T ...

Mas SysTick pode aparecer intercalado dependendo do momento do tick e das prioridades escolhidas.

✅ **O ponto principal:**

- SVC aparece imediatamente depois de 'c' (trap)
- PendSV aparece depois (deferido, low priority)

---

6) Dicas para o leitor “ver isso no Disassembly”

No Disassembly, procure:

- svc 0x01 (instrução geradora)
- prólogo/epílogo em SVC_Handler e PendSV_Handler
- SCB->ICSR escrita com bit 28 (pend)
- SysTick->LOAD/CTRL sendo configurados

A seguir vamos fechar a parte “core-only” com EXC_RETURN: o valor “mágico” que aparece no LR quando você está dentro de uma exceção (ISR/handler). Isso é um dos conceitos mais importantes para entender como o Cortex-M volta da interrupção, qual stack está sendo usada (MSP/PSP) e por que o retorno “não é um BX LR normal”.

Em Cortex-M0+ (ARMv6-M), o mecanismo de exception return e o frame empilhado são parte do modelo de exceções do Cortex-M.

---

1) O que é EXC_RETURN

Quando o Cortex-M entra em uma exceção (SysTick, PendSV, HardFault, IRQ externa etc.), ele:

1. empilha automaticamente o stack frame (r0–r3, r12, lr, pc, xPSR)
2. coloca no LR um valor especial chamado EXC_RETURN

Esse valor não é um endereço de retorno “normal”. Ele é um código que diz para o core:

- “quando fizer o return, volte para thread mode ou handler mode”
- “use MSP ou PSP para desempilhar”
- “retorne para o modo correto e restaure o contexto automaticamente”

Ou seja:

em exceções, BX LR vira “exception return”, e o hardware entende EXC_RETURN e desfaz o stacking.

---

2) Quais informações EXC_RETURN carrega

O EXC_RETURN é um valor no LR com padrão 0xFFFFFFxx. Em Cortex-M (visão geral):

- um dos bits indica se o retorno vai usar MSP ou PSP
- outro indica se volta para Thread mode ou Handler mode
- em cores com FPU há bits para frame estendido (não é o caso do M0+)

✅ **Para nós o mais importante é:**

- MSP vs PSP
- Thread vs Handler

---

3) O que você deve observar no debugger

Dentro de um handler (ex.: SysTick_Handler):

- LR (registrador R14) não parece um endereço
- LR tem cara de 0xFFFFFFF9, 0xFFFFFFFD etc.
- esse valor muda conforme você usa MSP/PSP e conforme o modo

✅ **Em bare-metal sem RTOS:**

- normalmente tudo roda em MSP
- então o LR/EXC_RETURN tende a ser sempre o mesmo padrão

Com RTOS:

- thread mode usa PSP e handlers usam MSP
- e o EXC_RETURN muda (você vê isso nitidamente)

---

4) Exemplo testável: capturar LR (EXC_RETURN) dentro de handlers

Vamos criar:

- SysTick_Handler e PendSV_Handler capturando:

o LR (EXC_RETURN)

o MSP, PSP

o CONTROL e IPSR

Depois vamos:

- forçar PendSV

- deixar SysTick rodar
- inspecionar as variáveis no debugger

4.1 exc_return_demo.h

```
#ifndef EXC_RETURN_DEMO_H
#define EXC_RETURN_DEMO_H

#include <stdint.h>

void exc_return_demo_run(void);

/* Observáveis */
extern volatile uint32_t g_lr_systick;
extern volatile uint32_t g_lr_pendsv;

extern volatile uint32_t g_msp_systick;
extern volatile uint32_t g_psp_systick;
extern volatile uint32_t g_control_systick;
extern volatile uint32_t g_ipsr_systick;

extern volatile uint32_t g_msp_pendsv;
extern volatile uint32_t g_psp_pendsv;
extern volatile uint32_t g_control_pendsv;
extern volatile uint32_t g_ipsr_pendsv;

#endif
```

4.2 exc_return_demo.c

```
#include "exc_return_demo.h"
#include "stm32g0xx.h"
```

```
#include "core_cm0plus.h"

volatile uint32_t g_lr_systick = 0;
volatile uint32_t g_lr_pendsv  = 0;

volatile uint32_t g_msp_systick = 0;
volatile uint32_t g_psp_systick = 0;
volatile uint32_t g_control_systick = 0;
volatile uint32_t g_ipsr_systick = 0;

volatile uint32_t g_msp_pendsv = 0;
volatile uint32_t g_psp_pendsv = 0;
volatile uint32_t g_control_pendsv = 0;
volatile uint32_t g_ipsr_pendsv = 0;

static void systick_config_1ms(void)
{
    uint32_t ticks = (SystemCoreClock / 1000u) - 1u;

    SysTick->CTRL = 0;
    SysTick->LOAD = ticks;
    SysTick->VAL  = 0;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk   |
                    SysTick_CTRL_ENABLE_Msk;
}

/* Captura LR usando asm (porque LR não é acessível via intrinsic direto) */
```

```
static inline uint32_t read_lr(void)
{
    uint32_t v;
    __asm volatile ("mov %0, lr" : "=r"(v));
    return v;
}

void SysTick_Handler(void)
{
    g_lr_systick      = read_lr();
    g_msp_systick     = __get_MSP();
    g_psp_systick     = __get_PSP();
    g_control_systick = __get_CONTROL();
    g_ipsr_systick    = __get_IPSR();
}

void PendSV_Handler(void)
{
    g_lr_pendsv      = read_lr();
    g_msp_pendsv     = __get_MSP();
    g_psp_pendsv     = __get_PSP();
    g_control_pendsv = __get_CONTROL();
    g_ipsr_pendsv    = __get_IPSR();

    __BKPT(0); /* pare aqui para inspecionar imediatamente */
}

void exc_return_demo_run(void)
```

```
{
   /* Ambiente típico bare-metal: CONTROL/SPSEL = 0 => MSP em thread mode */
   systick_config_1ms();

   __NOP(); /* breakpoint A: espere 1-2ms, SysTick_Handler deve rodar */

   /* Força PendSV para inspecionar EXC_RETURN */
   SCB->ICSR = (1u << 28); /* PENDSVSET */

   __NOP(); /* breakpoint B: só chega aqui depois do PendSV (após continuar do BKPT)
*/
}
```

4.3 Chame no main.c

```
#include "exc_return_demo.h"

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  exc_return_demo_run();

  while (1) { }
}
```

---

5) Como interpretar o resultado

Quando o debugger parar no BKPT do PendSV_Handler, observe:

- g_lr_pendsv e g_lr_systick

→ valores do tipo 0xFFFFFFxx (EXC_RETURN)

- g_ipsr_pendsv

→ não é 0 (está em handler mode)

- g_msp_* e g_psp_*

→ em bare-metal, PSP geralmente é 0 ou não usado

→ MSP é válido e muda ao entrar em exceção (stack cresce)

- g_control_*

→ normalmente 0 (thread privileged + MSP)

✅ **Conclusão do leitor:**

- o LR em exceções não é endereço: é EXC_RETURN
- o core usa esse valor para saber como desempilhar e retornar

---

6) Extra testável: mudar Thread Mode para usar PSP (sem RTOS)

Agora vamos fazer o leitor enxergar a mudança do EXC_RETURN na prática:

1. cria um array psp_stack[]
2. aponta PSP para o topo desse array
3. muda CONTROL para usar PSP no thread mode (SPSEL=1)
4. dispara PendSV e observa que EXC_RETURN muda

⚠️ **Isso é avançado, mas é muito didático.**

Adicione isto ao exc_return_demo.c (opcional)

```
static uint32_t psp_stack[128];


static void switch_thread_to_psp(void)
{
  /* PSP deve apontar para o topo (stack cresce para baixo) */
  uint32_t top = (uint32_t)&psp_stack[128];


  __set_PSP(top);
```

```
  /* CONTROL bit1 (SPSEL) = 1 => thread mode usa PSP */
  uint32_t ctrl = __get_CONTROL();
  ctrl |= (1u << 1);
  __set_CONTROL(ctrl);

  __ISB(); /* sincroniza pipeline após mudar CONTROL */
}
```

E no exc_return_demo_run():

```
switch_thread_to_psp();
SCB->ICSR = (1u << 28); /* pend pendsv */
```

✅ **O que o leitor verá:**

- g_psp_pendsv agora será “real”
- g_lr_pendsv tende a mudar para o padrão de “return using PSP” (EXC_RETURN diferente)

Isso prova que:

- EXC_RETURN codifica o stack usado no retorno.

---

7) Por que isso importa no mundo real (firmware)

- RTOS usa PSP para tasks, MSP para handlers
- PendSV costuma ser usado para context switch (prioridade baixa)
- entender EXC_RETURN ajuda a:
  - debugar travas em ISR
  - entender stack overflow em tasks
  - interpretar dumps de hardfault e backtrace

A seguir vamos demonstrar, de forma testável, o stack frame empilhado automaticamente pelo Cortex-M0+ ao entrar em uma exceção (SysTick/PendSV), e comparar com o que fizemos no HardFault dump.

A ideia é você enxergar no debugger:

- o endereço do stack frame (SP no início do handler)
- os 8 words empilhados: r0,r1,r2,r3,r12,lr,pc,xpsr
- o PC empilhado apontando para a instrução “interrompida”
- a diferença entre MSP/PSP quando mudamos o stack do thread mode

Esse frame é parte do modelo de exceções Cortex-M (ARMv6-M).

---

1) O que vamos provar na prática

Quando uma exceção acontece, o hardware empilha sempre este frame (8 palavras, 32 bytes):

Índice	Conteúdo
0	r0
1	r1
2	r2
3	r3
4	r12
5	lr
6	pc
7	xpsr

E o SP dentro do handler aponta exatamente para &frame[0].

---

2) Código testável: capturar o frame em SysTick e PendSV

2.1 stack_frame_demo.h

```
#ifndef STACK_FRAME_DEMO_H

#define STACK_FRAME_DEMO_H


#include <stdint.h>
```

```
typedef struct
{
    uint32_t r0;
    uint32_t r1;
    uint32_t r2;
    uint32_t r3;
    uint32_t r12;
    uint32_t lr;
    uint32_t pc;
    uint32_t xpsr;
} hw_stack_frame_t;

/* roda o demo */
void stack_frame_demo_run(void);

/* observáveis */
extern volatile hw_stack_frame_t g_sf_systick;
extern volatile hw_stack_frame_t g_sf_pendsv;

extern volatile uint32_t g_sp_in_systick;
extern volatile uint32_t g_sp_in_pendsv;

extern volatile uint32_t g_msp_in_systick;
extern volatile uint32_t g_psp_in_systick;

extern volatile uint32_t g_msp_in_pendsv;
extern volatile uint32_t g_psp_in_pendsv;
```

```
extern volatile uint32_t g_main_marker;

#endif
```

2.2 stack_frame_demo.c

```
#include "stack_frame_demo.h"
#include "stm32g0xx.h"
#include "core_cm0plus.h"

volatile hw_stack_frame_t g_sf_systick;
volatile hw_stack_frame_t g_sf_pendsv;

volatile uint32_t g_sp_in_systick = 0;
volatile uint32_t g_sp_in_pendsv  = 0;

volatile uint32_t g_msp_in_systick = 0;
volatile uint32_t g_psp_in_systick = 0;

volatile uint32_t g_msp_in_pendsv = 0;
volatile uint32_t g_psp_in_pendsv = 0;

volatile uint32_t g_main_marker = 0;

/* stack opcional para PSP (para mostrar diferença MSP vs PSP) */
static uint32_t psp_stack[128];

static void systick_config_1ms(void)
{
   uint32_t ticks = (SystemCoreClock / 1000u) - 1u;
```

```
    SysTick->CTRL = 0;
    SysTick->LOAD = ticks;
    SysTick->VAL  = 0;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk   |
                    SysTick_CTRL_ENABLE_Msk;
}

/* Copia os 8 words do frame para estrutura observável */
static void copy_frame(volatile hw_stack_frame_t *dst, uint32_t *sp)
{
    dst->r0   = sp[0];
    dst->r1   = sp[1];
    dst->r2   = sp[2];
    dst->r3   = sp[3];
    dst->r12  = sp[4];
    dst->lr   = sp[5];
    dst->pc   = sp[6];
    dst->xpsr = sp[7];
}

/* Naked handlers para capturar SP “exato” do frame */
__attribute__((naked)) void SysTick_Handler(void)
{
    __asm volatile(
        "mov r0, sp            \n" /* r0 = SP apontando para frame[0] */
        "b   SysTick_Handler_C   \n"
```

```
    );
}

void SysTick_Handler_C(uint32_t *sp)
{
    g_sp_in_systick  = (uint32_t)sp;
    g_msp_in_systick = __get_MSP();
    g_psp_in_systick = __get_PSP();

    copy_frame(&g_sf_systick, sp);
    /* não trava aqui para deixar o demo rodar */
}

__attribute__((naked)) void PendSV_Handler(void)
{
    __asm volatile(
        "mov r0, sp             \n"
        "b   PendSV_Handler_C    \n"
    );
}

void PendSV_Handler_C(uint32_t *sp)
{
    g_sp_in_pendsv  = (uint32_t)sp;
    g_msp_in_pendsv = __get_MSP();
    g_psp_in_pendsv = __get_PSP();

    copy_frame(&g_sf_pendsv, sp);
```

```
    __BKPT(0); /* pare aqui para analisar o frame no debugger */
}

/* Opcional: faz thread mode usar PSP para o leitor ver diferença */
static void switch_thread_to_psp(void)
{
    uint32_t top = (uint32_t)&psp_stack[128];

    __set_PSP(top);

    uint32_t ctrl = __get_CONTROL();
    ctrl |= (1u << 1);      /* SPSEL=1 => thread mode usa PSP */
    __set_CONTROL(ctrl);

    __ISB();
}

void stack_frame_demo_run(void)
{
    /* 1) coloque um marcador “onde estávamos no main”
       para o leitor relacionar com o PC empilhado */
    g_main_marker = 0xAABBCCDDu;

    /* 2) (Opcional) ativar PSP no thread mode para demonstrar */
    switch_thread_to_psp();

    /* 3) iniciar SysTick para gerar exceções */
```

```
    systick_config_1ms();

    /* 4) deixe SysTick rodar um pouco */
    for (volatile uint32_t i = 0; i < 200000u; i++)
        __NOP();

    /* 5) force PendSV (vai parar no BKPT do PendSV_Handler_C) */
    SCB->ICSR = (1u << 28); /* PENDSVSET */

    /* Após continuar do BKPT, cai aqui */
    __NOP();
}
```

2.3 Chame no main.c

```
#include "stack_frame_demo.h"

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  stack_frame_demo_run();

  while (1) { }
}
```

---

3) Como você analisa no debugger (passo a passo)

Quando parar no BKPT(0) em PendSV_Handler_C:

3.1 Ver o endereço do frame

- g_sp_in_pendsv é o endereço do frame empilhado
- g_sf_pendsv contém a cópia dos campos

✅ **Isso prova que SP no início do handler aponta para o frame.**

3.2 O "ouro": pc empilhado

Veja:

- g_sf_pendsv.pc

Esse valor é o endereço da instrução que estava prestes a executar quando a exceção entrou (o ponto interrompido).

No Disassembly, você pode ir para esse PC e ver exatamente "onde estava".

3.3 Conferir xpsr

- g_sf_pendsv.xpsr contém flags e o IPSR embutido

3.4 Conferir lr empilhado

- g_sf_pendsv.lr é o LR da thread interrompida (não confundir com o LR "EXC_RETURN" do handler)

✅ **Isso é uma confusão comum:**

- LR dentro do frame = LR do código interrompido
- LR do handler = EXC_RETURN

---

4) Comparação com HardFault dump

O mecanismo de captura é o mesmo:

- no HardFault dump, usamos o SP para copiar os 8 words
- aqui fazemos isso em SysTick/PendSV

Diferença prática:

- no HardFault você geralmente quer parar e investigar o PC faulting
- em SysTick/PendSV isso é didático para entender o mecanismo de exceção e retorno

---

5) Extra: provando MSP vs PSP

Como chamamos switch_thread_to_psp(), o leitor deve observar:

- Em thread mode: CONTROL.SPSEL=1 e o código “normal” usa PSP
- Ao entrar em exceções, muitos sistemas usam MSP (depende do core/uso), mas:
  - __get_PSP() agora mostra um valor válido (topo do psp_stack)
  - __get_MSP() continua válido para handlers

✅ **O que o leitor aprende:**

- PSP pode ser configurado e usado para “thread mode” (base para RTOS)
- o core salva/restaura contexto a partir do stack atual do modo interrompido

---

6) Checklist

- Eu consigo localizar o pc empilhado e ir nele no disassembly.
- Eu entendo que o frame empilhado é sempre 8 words (sem FPU no M0+).
- Eu sei diferenciar LR empilhado vs LR(EXC_RETURN) do handler.
- Eu consigo ver PSP ativo ao mudar CONTROL.SPSEL.

Vamos transformar o “dump do frame” em uma biblioteca core-only reutilizável, que você pode chamar de HardFault, PendSV, SysTick, assert handler, etc. Ela:

- recebe sp (ponteiro para o frame empilhado)
- copia o frame para uma struct
- captura registradores úteis do core (MSP/PSP/CONTROL/PRIMASK/IPSR/VTOR)
- deixa tudo em variáveis globais fáceis de inspecionar no debugger

---

1) Biblioteca: core_exc_dump

core_exc_dump.h (Core/Inc)

```
#ifndef CORE_EXC_DUMP_H

#define CORE_EXC_DUMP_H


#include <stdint.h>
```

```
/* Frame empilhado automaticamente pelo hardware em exceções (M0+) */
typedef struct
{
    uint32_t r0;
    uint32_t r1;
    uint32_t r2;
    uint32_t r3;
    uint32_t r12;
    uint32_t lr;     /* LR do código interrompido */
    uint32_t pc;     /* PC do código interrompido */
    uint32_t xpsr;
} core_hw_frame_t;

/* Snapshot extra do core no momento do dump */
typedef struct
{
    uint32_t msp;
    uint32_t psp;
    uint32_t primask;
    uint32_t control;
    uint32_t ipsr;
    uint32_t vtor;
    uint32_t exc_return_lr; /* LR do handler (EXC_RETURN), se capturado */
    uint32_t sp_at_entry;   /* SP passado para o dump (endereço do frame) */
} core_exc_snapshot_t;

/* Dump completo (frame + snapshot) */
typedef struct
```

```
{
    core_hw_frame_t      frame;
    core_exc_snapshot_t  snap;
} core_exc_dump_t;

/* Dump global (observável no debugger) */
extern volatile core_exc_dump_t g_core_exc_dump;

/* Função principal: copia frame + captura contexto */
void core_exc_dump_capture(uint32_t *sp, uint32_t exc_return_lr);

/* Função auxiliar: limpa o dump (opcional) */
void core_exc_dump_clear(void);

#endif /* CORE_EXC_DUMP_H */
```

---

core_exc_dump.c (Core/Src)

```
#include "core_exc_dump.h"
#include "stm32g0xx.h"
#include "core_cm0plus.h"

volatile core_exc_dump_t g_core_exc_dump;

void core_exc_dump_clear(void)
{
    /* Zera tudo manualmente para evitar libc/memset (didático e core-only) */
    volatile uint32_t *p = (volatile uint32_t *)&g_core_exc_dump;
    for (uint32_t i = 0; i < (sizeof(g_core_exc_dump) / sizeof(uint32_t)); i++)
```

```
        p[i] = 0;
}

static void copy_frame(volatile core_hw_frame_t *dst, uint32_t *sp)
{
    dst->r0   = sp[0];
    dst->r1   = sp[1];
    dst->r2   = sp[2];
    dst->r3   = sp[3];
    dst->r12  = sp[4];
    dst->lr   = sp[5];
    dst->pc   = sp[6];
    dst->xpsr = sp[7];
}

void core_exc_dump_capture(uint32_t *sp, uint32_t exc_return_lr)
{
    /* Salva SP do frame e LR do handler (EXC_RETURN) */
    g_core_exc_dump.snap.sp_at_entry    = (uint32_t)sp;
    g_core_exc_dump.snap.exc_return_lr  = exc_return_lr;

    /* Salva contexto do core */
    g_core_exc_dump.snap.msp     = __get_MSP();
    g_core_exc_dump.snap.psp     = __get_PSP();
    g_core_exc_dump.snap.primask = __get_PRIMASK();
    g_core_exc_dump.snap.control = __get_CONTROL();
    g_core_exc_dump.snap.ipsr    = __get_IPSR();
    g_core_exc_dump.snap.vtor    = SCB->VTOR;
```

```
    /* Copia frame empilhado */
    copy_frame(&g_core_exc_dump.frame, sp);
}
```

---

2) Como usar em handlers (padrão pronto)

A ideia é sempre:

1. “pegar SP e LR do handler” (LR = EXC_RETURN)
2. chamar core_exc_dump_capture(sp, lr)
3. parar com BKPT para inspeção

2.1 HardFault usando biblioteca (Core/Src/hardfault_handler.c)

```
#include "core_exc_dump.h"
#include "core_cm0plus.h"

void HardFault_Handler_C(uint32_t *sp, uint32_t exc_lr)
{
    core_exc_dump_capture(sp, exc_lr);
    __BKPT(0);
    while (1) { __NOP(); }
}

__attribute__((naked)) void HardFault_Handler(void)
{
    __asm volatile(
        "mov r0, sp       \n" /* r0 = sp */
        "mov r1, lr       \n" /* r1 = exc_return (LR do handler) */
        "b   HardFault_Handler_C \n"
    );
```

```
}
```

✅ **Agora o HardFault dump fica padronizado e reutilizável.**

2.2 PendSV usando biblioteca (Core/Src/pendsv_handler.c)

```
#include "core_exc_dump.h"

#include "core_cm0plus.h"


void PendSV_Handler_C(uint32_t *sp, uint32_t exc_lr)

{

   core_exc_dump_capture(sp, exc_lr);

   __BKPT(0);

}


__attribute__((naked)) void PendSV_Handler(void)

{

   __asm volatile(

      "mov r0, sp \n"

      "mov r1, lr \n"

      "b   PendSV_Handler_C \n"

   );

}
```

2.3 SysTick usando biblioteca (sem travar)

SysTick acontece o tempo todo, então não queremos BKPT sempre. Podemos capturar apenas quando solicitado.

systick_handler.c

```
#include "core_exc_dump.h"

#include "core_cm0plus.h"
```

```
#include <stdint.h>

volatile uint32_t g_systick_ticks = 0;
volatile uint32_t g_capture_next_systick = 0;

void SysTick_Handler_C(uint32_t *sp, uint32_t exc_lr)
{
    g_systick_ticks++;

    if (g_capture_next_systick)
    {
        g_capture_next_systick = 0;
        core_exc_dump_capture(sp, exc_lr);
        __BKPT(0);
    }
}

__attribute__((naked)) void SysTick_Handler(void)
{
    __asm volatile(
        "mov r0, sp \n"
        "mov r1, lr \n"
        "b   SysTick_Handler_C \n"
    );
}
```

✅ **Você pode setar g_capture_next_systick = 1 no debugger e capturar "o próximo tick".**

---

3) Exemplo testável: capturar frame em SysTick e depois forçar PendSV

core_exc_dump_demo.c (Core/Src)

```
#include "stm32g0xx.h"
#include "core_cm0plus.h"
#include "core_exc_dump.h"
#include <stdint.h>

extern volatile uint32_t g_capture_next_systick;

static void systick_config_1ms(void)
{
    uint32_t ticks = (SystemCoreClock / 1000u) - 1u;

    SysTick->CTRL = 0;
    SysTick->LOAD = ticks;
    SysTick->VAL  = 0;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk   |
                    SysTick_CTRL_ENABLE_Msk;
}

void core_exc_dump_demo_run(void)
{
    core_exc_dump_clear();

    systick_config_1ms();

    /* 1) pedir captura no próximo SysTick */
```

```
    g_capture_next_systick = 1;

    /* 2) esperar - vai parar no BKPT dentro do SysTick_Handler_C */
    for (volatile uint32_t i = 0; i < 500000u; i++)
        __NOP();

    /* 3) depois disso, force PendSV para capturar outro frame */
    SCB->ICSR = (1u << 28); /* PENDSVSET */
    __NOP(); /* aqui só chega após continuar do BKPT do PendSV */
}
```

No main.c:

```
extern void core_exc_dump_demo_run(void);

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  core_exc_dump_demo_run();

  while (1) { }
}
```

---

4) Como você usa o dump no debugger (checklist)

Quando o debugger parar em um BKPT (SysTick ou PendSV ou HardFault), o leitor abre o Watch:

- g_core_exc_dump.snap.ipsr
  - identifica a exceção ativa (não é 0)

- g_core_exc_dump.snap.exc_return_lr
  - EXC_RETURN (LR do handler)
- g_core_exc_dump.snap.sp_at_entry
  - endereço do frame
- g_core_exc_dump.frame.pc
  - endereço do código interrompido (ou faulting no HardFault)
- g_core_exc_dump.frame.lr
  - LR do código interrompido

✅ **E o mais didático:**

- copie o valor de frame.pc e use “Go to address” no Disassembly.

Esse padrão é exatamente o tipo de ferramenta que engenheiros usam em firmware real:

- você padroniza o dump de exceções
- reduz esforço de debug
- consegue investigar crash sem periféricos
- a mesma biblioteca serve para HardFault/PendSV/SysTick/assert

---

Vamos deixar essa biblioteca nível “produto”: salvar o dump em uma região .noinit (RAM que não é zerada no reset), e então resetar automaticamente após capturar a exceção. Depois do reboot, você consegue abrir o debugger e ver qual foi o PC do crash mesmo sem UART/log.

A receita tem 3 partes:

1. Criar uma seção .noinit no linker script (STM32CubeIDE/GCC)
2. Colocar o dump dentro dessa seção (__attribute__((section(".noinit"))))
3. Garantir que o startup não zere essa seção (ele não zera automaticamente se ela não estiver em .bss)
4. (Opcional) Guardar um “magic” para saber se o dump é válido.

---

1) Linker script: adicionar a seção .noinit

No STM32CubeIDE, o linker script fica normalmente em:

- STM32G070RBTX_FLASH.ld (ou nome parecido)

Abra o .ld e procure a área de RAM, algo como:

```
/* ... */
.bss :
{
  _sbss = .;
  *(.bss)
  *(.bss*)
  *(COMMON)
  _ebss = .;
} >RAM
```

Logo depois do .bss (ou antes, tanto faz, desde que seja em RAM), adicione:

```
/* Seção NOINIT: RAM que não é inicializada/zerada no reset */
.noinit (NOLOAD) :
{
  . = ALIGN(4);
  _snoinit = .;
  *(.noinit)
  *(.noinit*)
  . = ALIGN(4);
  _enoinit = .;
} >RAM
```

✅ **O que isso faz:**

- cria uma seção .noinit na RAM
- NOLOAD evita que o loader espere “dados iniciais” para ela
- como ela não está em .bss, o startup padrão não zera

Dica profissional: manter o .noinit alinhado a 4 bytes.

---

2) Definir o dump persistente em .noinit

Vamos criar uma versão do dump com:

- magic para indicar validade
- reason para indicar qual handler salvou (HardFault, PendSV etc.)
- dump com frame e snapshot

core_exc_persist.h (Core/Inc)

```
#ifndef CORE_EXC_PERSIST_H
#define CORE_EXC_PERSIST_H

#include <stdint.h>
#include "core_exc_dump.h"

/* “Magic” para identificar dump válido após reset */
#define EXC_PERSIST_MAGIC  (0x45584331u) /* 'EXC1' */

/* Tipos de origem */
#define EXC_SRC_NONE       0u
#define EXC_SRC_HARDFAULT  1u
#define EXC_SRC_PENDSV     2u
#define EXC_SRC_SYSTICK    3u
#define EXC_SRC_NMI        4u
#define EXC_SRC_SVC        5u

typedef struct
{
   uint32_t magic;
   uint32_t src;
```

```
    uint32_t counter;  /* incrementa a cada captura, útil para ver se regravou */
    core_exc_dump_t dump;
} core_exc_persist_t;

/* Dump persistente em RAM .noinit */
extern volatile core_exc_persist_t g_exc_persist;

/* API */
void core_exc_persist_clear(void);
int  core_exc_persist_is_valid(void);

void core_exc_persist_capture(uint32_t src, uint32_t *sp, uint32_t exc_return_lr);

/* Opcional: pedir reset após capturar */
void core_exc_persist_request_reset(void);

#endif /* CORE_EXC_PERSIST_H */
```

core_exc_persist.c (Core/Src)

```
#include "core_exc_persist.h"
#include "stm32g0xx.h"
#include "core_cm0plus.h"

/* Coloca na seção .noinit (precisa existir no linker script) */
volatile core_exc_persist_t g_exc_persist __attribute__((section(".noinit")));

void core_exc_persist_clear(void)
{
    /* Não use memset para manter “core-only”, mas poderia. */
```

```
    volatile uint32_t *p = (volatile uint32_t *)&g_exc_persist;
    for (uint32_t i = 0; i < (sizeof(g_exc_persist) / sizeof(uint32_t)); i++)
        p[i] = 0;
}

int core_exc_persist_is_valid(void)
{
    return (g_exc_persist.magic == EXC_PERSIST_MAGIC) ? 1 : 0;
}

void core_exc_persist_capture(uint32_t src, uint32_t *sp, uint32_t exc_return_lr)
{
    /* Marcar validade antes e depois é uma boa prática.
       Aqui fazemos simples: gravamos magic e payload. */
    g_exc_persist.magic = EXC_PERSIST_MAGIC;
    g_exc_persist.src   = src;
    g_exc_persist.counter++;

    /* Captura frame+snapshot usando a biblioteca anterior */
    core_exc_dump_capture(sp, exc_return_lr);

    /* Copia o dump global para o persistente */
    g_exc_persist.dump = g_core_exc_dump;
}

void core_exc_persist_request_reset(void)
{
    /* Reset via AIRCR: SYSRESETREQ com chave */
```

```
    __DSB();
    SCB->AIRCR = (0x5FAu << 16) | (1u << 2);
    __DSB();
    while (1) { __NOP(); }
}
```

---

3) Usar no HardFault_Handler (captura + reset automático)

hardfault_persist_handler.c (Core/Src)

```
#include "core_exc_persist.h"
#include "core_cm0plus.h"

void HardFault_Handler_C(uint32_t *sp, uint32_t exc_lr)
{
    core_exc_persist_capture(EXC_SRC_HARDFAULT, sp, exc_lr);

    /* Opcional: parar aqui quando estiver em debug */
    __BKPT(0);

    /* Em “modo produto”, você normalmente resetaria */
    core_exc_persist_request_reset();
}

__attribute__((naked)) void HardFault_Handler(void)
{
    __asm volatile(
        "mov r0, sp \n"
        "mov r1, lr \n"
        "b   HardFault_Handler_C \n"
```

```
    );
}
```

✅ **Resultado:**

- Ao ocorrer HardFault, você salva o dump na RAM .noinit
- E reseta
- Após o reboot, o dump continua lá

---

4) Mostrar o dump após o reboot (no main())

No main.c, logo após init do clock:

```
#include "core_exc_persist.h"

/* Variáveis “amigáveis” para o leitor olhar */
volatile uint32_t g_last_fault_valid = 0;
volatile uint32_t g_last_fault_src   = 0;
volatile uint32_t g_last_fault_pc    = 0;
volatile uint32_t g_last_fault_lr    = 0;
volatile uint32_t g_last_fault_ipsr  = 0;

static void check_last_fault(void)
{
    if (core_exc_persist_is_valid())
    {
        g_last_fault_valid = 1;
        g_last_fault_src   = g_exc_persist.src;
        g_last_fault_pc    = g_exc_persist.dump.frame.pc;
        g_last_fault_lr    = g_exc_persist.dump.frame.lr;
        g_last_fault_ipsr  = g_exc_persist.dump.snap.ipsr;
```

```
        /* Se quiser “consumir” o dump depois de ler: */
        /* core_exc_persist_clear(); */
    }
    else
    {
        g_last_fault_valid = 0;
    }
}

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  check_last_fault();

  while (1) { }
}
```

✅ **Como você valida:**

- Rode normal: g_last_fault_valid deve ser 0
- Force um HardFault (ex.: call para ponteiro inválido)
- O MCU reseta
- Depois do reset, g_last_fault_valid vira 1
- g_last_fault_pc mostra o local do crash anterior

---

5) Teste prático: forçar HardFault para ver persistência

Crie um arquivo fault_trigger.c:

```
#include <stdint.h>
```

```
void fault_trigger_run(void)
{
   /* Função inválida -> HardFault */
   void (*bad_fn)(void) = (void (*)(void))0x00000001u;
   bad_fn();
}
```

Exemplo de chamada no `main()` após `check_last_fault()`:

```
extern void fault_trigger_run(void);

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  check_last_fault();

  /* Descomente para disparar o fault: */
  // fault_trigger_run();

  while (1) { }
}
```

Seu procedimento:

1. Rode com fault_trigger_run() comentado → g_last_fault_valid = 0
2. Descomente fault_trigger_run() → dá HardFault → salva dump → reset
3. Comente de novo fault_trigger_run() e rode → agora g_last_fault_valid = 1 e g_last_fault_pc é o PC do fault anterior

---

6) Observações profissionais importantes

A) “Por que isso funciona?”

Porque:

- .noinit não está em .bss
- o startup só zera .bss e copia .data
- então .noinit fica intacta no reset

B) Quando o conteúdo pode se perder?

- power cycle (sem alimentação, RAM perde)
- certos modos de low-power podem apagar RAM (isso é periférico ST e veremos depois)
- reset que reinicialize RAM por hardware (normalmente não é o caso)

C) Tamanho e cuidado

- não faça dumps gigantes em .noinit
- mantenha alinhamento e organização

A seguir vão dois upgrades profissionais para o dump persistente em .noinit:

1. CRC simples para validar integridade do dump (detectar RAM corrompida).
2. Histórico (ring buffer) com os últimos N faults (ex.: 4), para não perder o crash anterior quando outro acontece.

1) Upgrade: CRC simples do dump persistente

1.1 O que vamos fazer

- Calcular um CRC “didático” (na prática, checksum) sobre o conteúdo do dump.
- Guardar crc junto com os dados.
- Na leitura após reset, validar:

o magic ok

o crc confere

✅ **CRC simples sugerido (bem robusto para didática):**

- CRC-32 não (exigiria mais código)

- Vamos usar um checksum 32-bit com mistura (XOR + rotação) para melhorar.

---

2) Upgrade: histórico em ring buffer (últimos N faults)

2.1 Estrutura

Criamos um array entries[N] em .noinit e um índice head:

- quando um fault ocorre:
  - grava em entries[head]
  - head = (head + 1) % N
- cada entrada guarda:
  - src (HardFault, NMI, etc.)
  - counter / seq
  - dump completo
  - crc

---

3) Código completo (pronto para copiar)

3.1 core_exc_persist2.h (Core/Inc)

```
#ifndef CORE_EXC_PERSIST2_H
#define CORE_EXC_PERSIST2_H

#include <stdint.h>
#include "core_exc_dump.h"

#define EXC_PERSIST_MAGIC  (0x45584332u) /* 'EXC2' */

/* fontes */
#define EXC_SRC_NONE       0u
#define EXC_SRC_HARDFAULT  1u
#define EXC_SRC_PENDSV     2u
```

```
#define EXC_SRC_SYSTICK    3u
#define EXC_SRC_NMI        4u
#define EXC_SRC_SVC        5u

/* quantos faults manter */
#define EXC_HISTORY_N      4u

typedef struct
{
    uint32_t magic;
    uint32_t src;
    uint32_t seq;        /* contador crescente */
    uint32_t crc;        /* checksum do payload */
    core_exc_dump_t dump;
} core_exc_entry_t;

typedef struct
{
    uint32_t magic;      /* EXC_PERSIST_MAGIC */
    uint32_t head;       /* próximo índice de escrita */
    uint32_t seq;        /* seq global */
    uint32_t reserved;

    core_exc_entry_t entries[EXC_HISTORY_N];
} core_exc_history_t;

/* Histórico persistente em .noinit */
extern volatile core_exc_history_t g_exc_hist;
```

```
/* API */
void core_exc_hist_clear(void);
int  core_exc_hist_is_valid(void);

/* Captura + grava no ring (com checksum) */
void core_exc_hist_capture(uint32_t src, uint32_t *sp, uint32_t exc_return_lr);

/* Valida uma entrada específica (magic + crc) */
int  core_exc_hist_entry_valid(uint32_t index);

/* Reset do sistema (opcional) */
void core_exc_hist_request_reset(void);

#endif /* CORE_EXC_PERSIST2_H */
```

3.2 core_exc_persist2.c (Core/Src)

```
#include "core_exc_persist2.h"
#include "stm32g0xx.h"
#include "core_cm0plus.h"

/* Histórico em RAM persistente */
volatile core_exc_history_t g_exc_hist __attribute__((section(".noinit")));

/* ----------------------------
 * checksum simples (32-bit)
 * ---------------------------- */
static uint32_t rotl32(uint32_t x, uint32_t r)
```

```
{
    return (x << r) | (x >> (32u - r));
}

static uint32_t checksum32(const void *data, uint32_t bytes)
{
    /* Mistura simples: soma + xor + rotação */
    const uint8_t *p = (const uint8_t *)data;
    uint32_t acc = 0xA5A5A5A5u;

    for (uint32_t i = 0; i < bytes; i++)
    {
        acc ^= p[i];
        acc += (uint32_t)p[i] * 0x01000193u;
        acc = rotl32(acc, 5);
    }
    return acc;
}

/* calcula checksum do payload “dump”, não inclui magic/src/seq/crc */
static uint32_t entry_compute_crc(const core_exc_entry_t *e)
{
    return checksum32(&e->dump, (uint32_t)sizeof(e->dump));
}

void core_exc_hist_clear(void)
{
    volatile uint32_t *p = (volatile uint32_t *)&g_exc_hist;
```

```
    for (uint32_t i = 0; i < (sizeof(g_exc_hist) / sizeof(uint32_t)); i++)
        p[i] = 0;
}

int core_exc_hist_is_valid(void)
{
    return (g_exc_hist.magic == EXC_PERSIST_MAGIC) ? 1 : 0;
}

int core_exc_hist_entry_valid(uint32_t index)
{
    if (index >= EXC_HISTORY_N) return 0;

    const core_exc_entry_t *e = (const core_exc_entry_t *)&g_exc_hist.entries[index];
    if (e->magic != EXC_PERSIST_MAGIC) return 0;

    uint32_t crc = entry_compute_crc(e);
    return (crc == e->crc) ? 1 : 0;
}

void core_exc_hist_capture(uint32_t src, uint32_t *sp, uint32_t exc_return_lr)
{
    /* inicializa header do histórico se for a primeira vez */
    if (g_exc_hist.magic != EXC_PERSIST_MAGIC)
    {
        g_exc_hist.magic = EXC_PERSIST_MAGIC;
        g_exc_hist.head  = 0;
        g_exc_hist.seq   = 0;
```

```
    }

    /* Captura dump “atual” para o buffer global */
    core_exc_dump_capture(sp, exc_return_lr);

    /* Decide slot */
    uint32_t idx = g_exc_hist.head;
    if (idx >= EXC_HISTORY_N) idx = 0;

    /* Preenche entrada */
    core_exc_entry_t *e = (core_exc_entry_t *)&g_exc_hist.entries[idx];
    e->magic = EXC_PERSIST_MAGIC;
    e->src   = src;
    e->seq   = ++g_exc_hist.seq;
    e->dump  = g_core_exc_dump;

    /* Calcula CRC do payload e grava */
    e->crc = entry_compute_crc(e);

    /* Avança ring */
    g_exc_hist.head = (idx + 1u) % EXC_HISTORY_N;
}

void core_exc_hist_request_reset(void)
{
    __DSB();
    SCB->AIRCR = (0x5FAu << 16) | (1u << 2); /* SYSRESETREQ */
    __DSB();
```

```
    while (1) { __NOP(); }
}
```

---

4) Usar no HardFault (captura + reset + histórico)

hardfault_hist_handler.c (Core/Src)

```
#include "core_exc_persist2.h"
#include "core_cm0plus.h"

void HardFault_Handler_C(uint32_t *sp, uint32_t exc_lr)
{
    core_exc_hist_capture(EXC_SRC_HARDFAULT, sp, exc_lr);

    /* Debug: pare aqui para olhar o dump antes do reset */
    __BKPT(0);

    /* Produto: reset automático */
    core_exc_hist_request_reset();
}

__attribute__((naked)) void HardFault_Handler(void)
{
    __asm volatile(
        "mov r0, sp \n"
        "mov r1, lr \n"
        "b   HardFault_Handler_C \n"
    );
}
```

---

5) Ler o histórico após reset (no main.c)

Crie variáveis amigáveis para o leitor:

```
#include "core_exc_persist2.h"

volatile uint32_t g_hist_valid = 0;
volatile uint32_t g_hist_head  = 0;

volatile uint32_t g_last0_valid = 0;
volatile uint32_t g_last0_src   = 0;
volatile uint32_t g_last0_seq   = 0;
volatile uint32_t g_last0_pc    = 0;

volatile uint32_t g_last1_valid = 0;
volatile uint32_t g_last1_src   = 0;
volatile uint32_t g_last1_seq   = 0;
volatile uint32_t g_last1_pc    = 0;

static void load_fault_history_summary(void)
{
   g_hist_valid = core_exc_hist_is_valid();
   if (!g_hist_valid) return;

   g_hist_head = g_exc_hist.head;

   /* Vamos ler duas entradas mais recentes:
      last0 = última gravada
      last1 = penúltima
    */
```

```
    uint32_t last0 = (g_hist_head + EXC_HISTORY_N - 1u) % EXC_HISTORY_N;
    uint32_t last1 = (g_hist_head + EXC_HISTORY_N - 2u) % EXC_HISTORY_N;

    g_last0_valid = core_exc_hist_entry_valid(last0);
    if (g_last0_valid)
    {
        g_last0_src = g_exc_hist.entries[last0].src;
        g_last0_seq = g_exc_hist.entries[last0].seq;
        g_last0_pc  = g_exc_hist.entries[last0].dump.frame.pc;
    }

    g_last1_valid = core_exc_hist_entry_valid(last1);
    if (g_last1_valid)
    {
        g_last1_src = g_exc_hist.entries[last1].src;
        g_last1_seq = g_exc_hist.entries[last1].seq;
        g_last1_pc  = g_exc_hist.entries[last1].dump.frame.pc;
    }
}
```

E chame no main() após clock:

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  load_fault_history_summary();

  while (1) { }
```

```
}
```

✅ **Você inspeciona:**

- g_last0_pc / g_last1_pc
- g_last0_src / g_last1_src (1 = HardFault)
- g_last0_seq (contador)

---

6) Teste prático: gerar múltiplos faults e ver o ring funcionar

Crie fault_trigger.c:

```
#include <stdint.h>


void fault_trigger_run_1(void)
{
   void (*bad_fn)(void) = (void (*)(void))0x00000001u;
   bad_fn();
}


void fault_trigger_run_2(void)
{
   /* Acesso inválido (pode faultar dependendo do endereço) */
   volatile uint32_t *p = (volatile uint32_t *)0xFFFFFFF0u;
   *p = 0x1234u;
}
```

No main, você alterna qual chamar (um por vez) e observa que:

- a cada reset, o head avança
- o seq cresce
- last0 e last1 mudam

Importante: sempre comente a chamada após validar, senão vai entrar em loop infinito de reset.

---

7) Lembrete: .noinit no linker script

Caso a secao ainda nao exista no linker script, adicione:

```
.noinit (NOLOAD) :
{
  . = ALIGN(4);
  _snoinit = .;
  *(.noinit)
  *(.noinit*)
  . = ALIGN(4);
  _enoinit = .;
} >RAM
```

---

1) Upgrade: adicionar reason_detail na entrada do histórico

1.1 O que é reason_detail

Um campo extra para guardar um “detalhe rápido” do motivo. Exemplos úteis:

- pc do crash (principal)
- lr do crash
- ipsr (qual exceção)
- exc_return (stack/return mode)

Para ficar simples e útil:

- reason_detail = pc (por padrão)
- e podemos guardar outros detalhes em campos extras.

1.2 Atualize core_exc_persist2.h

Substitua a struct core_exc_entry_t por esta versão:

```
typedef struct
```

```
{
    uint32_t magic;
    uint32_t src;
    uint32_t seq;

    uint32_t reason_detail; /* ex.: PC do crash */
    uint32_t crc;          /* checksum do payload */

    core_exc_dump_t dump;
} core_exc_entry_t;
```

Isso altera o layout e o checksum continua cobrindo apenas dump, então está ok.

1.3 Atualize core_exc_persist2.c (na captura)

Dentro de core_exc_hist_capture, após copiar o dump:

```
e->dump  = g_core_exc_dump;

/* detalhe rápido: PC do ponto interrompido/fault */
e->reason_detail = e->dump.frame.pc;

/* CRC do payload (dump) */
e->crc = entry_compute_crc(e);
```

✅ **Agora cada entrada tem um “resumo numérico” direto (reason_detail).**

---

2) Upgrade: “relatório” amigável no debugger (sem UART)

Vamos criar:

- uma função que traduz src em string ("HardFault", "NMI", etc.)
- uma estrutura de “view” que junta:

o const char *src_name

- seq
- pc
- lr
- ipsr
- exc_return
- vtor

- uma função core_exc_hist_build_view_last() que preenche essas variáveis globais.

Isso permite que o leitor abra o Watch e veja:

- g_view_src_name → “HardFault”
- g_view_pc → 0x0800....
- etc.

2.1 core_exc_view.h (Core/Inc)

```
#ifndef CORE_EXC_VIEW_H
#define CORE_EXC_VIEW_H

#include <stdint.h>

/* Atualiza a “view” com a última entrada válida do histórico */
void core_exc_view_update_last(void);

/* Atualiza a “view” com uma entrada específica do histórico */
void core_exc_view_update_index(uint32_t index);

/* Observáveis no debugger */
extern volatile uint32_t g_view_valid;
extern const char *g_view_src_name;
```

```
extern volatile uint32_t g_view_seq;
extern volatile uint32_t g_view_pc;
extern volatile uint32_t g_view_lr;
extern volatile uint32_t g_view_ipsr;
extern volatile uint32_t g_view_exc_return;
extern volatile uint32_t g_view_vtor;
extern volatile uint32_t g_view_sp_at_entry;
extern volatile uint32_t g_view_reason_detail;

#endif /* CORE_EXC_VIEW_H */
```

2.2 core_exc_view.c (Core/Src)

```
#include "core_exc_view.h"
#include "core_exc_persist2.h"

/* Observáveis */
volatile uint32_t g_view_valid = 0;
const char *g_view_src_name = 0;

volatile uint32_t g_view_seq = 0;
volatile uint32_t g_view_pc = 0;
volatile uint32_t g_view_lr = 0;
volatile uint32_t g_view_ipsr = 0;
volatile uint32_t g_view_exc_return = 0;
volatile uint32_t g_view_vtor = 0;
volatile uint32_t g_view_sp_at_entry = 0;
volatile uint32_t g_view_reason_detail = 0;
```

```
static const char* src_to_name(uint32_t src)
{
    switch (src)
    {
        case EXC_SRC_HARDFAULT: return "HardFault";
        case EXC_SRC_NMI:       return "NMI";
        case EXC_SRC_PENDSV:    return "PendSV";
        case EXC_SRC_SYSTICK:   return "SysTick";
        case EXC_SRC_SVC:       return "SVC";
        default:                return "Unknown";
    }
}

static void fill_view_from_entry(const core_exc_entry_t *e)
{
    g_view_valid = 1;
    g_view_src_name = src_to_name(e->src);

    g_view_seq = e->seq;

    g_view_reason_detail = e->reason_detail;

    g_view_pc  = e->dump.frame.pc;
    g_view_lr  = e->dump.frame.lr;

    g_view_ipsr = e->dump.snap.ipsr;
    g_view_exc_return = e->dump.snap.exc_return_lr;
```

```
    g_view_vtor = e->dump.snap.vtor;
    g_view_sp_at_entry = e->dump.snap.sp_at_entry;
}

void core_exc_view_update_index(uint32_t index)
{
    if (!core_exc_hist_is_valid())
    {
        g_view_valid = 0;
        g_view_src_name = "NoHistory";
        return;
    }

    if (!core_exc_hist_entry_valid(index))
    {
        g_view_valid = 0;
        g_view_src_name = "InvalidEntry";
        return;
    }

    const core_exc_entry_t *e = (const core_exc_entry_t *)&g_exc_hist.entries[index];
    fill_view_from_entry(e);
}

void core_exc_view_update_last(void)
{
    if (!core_exc_hist_is_valid())
```

```
  {
    g_view_valid = 0;
    g_view_src_name = "NoHistory";
    return;
  }

  uint32_t head = g_exc_hist.head;
  uint32_t last = (head + EXC_HISTORY_N - 1u) % EXC_HISTORY_N;

  core_exc_view_update_index(last);
}
```

---

3) Usar no main() (mostrar “relatório” após reboot)

No main.c:

```
#include "core_exc_view.h"
#include "core_exc_persist2.h"

int main(void)
{
 HAL_Init();
 SystemClock_Config();

 /* Monta a view para o debugger */
 core_exc_view_update_last();

 while (1) { }
}
```

✅ **Agora você abre Watch e vê:**

- g_view_valid
- g_view_src_name -> "HardFault"
- g_view_pc -> endereço do crash
- g_view_reason_detail -> mesmo PC (ou outro detalhe, se você mudar a política)
- g_view_seq -> número do evento
- g_view_exc_return -> EXC_RETURN
- g_view_sp_at_entry -> endereço do frame

---

4) Ajuste recomendado: “consumir” ou manter histórico?

Você escolhe:

- manter o histórico para sempre (bom para debug)
- ou “limpar depois de ler”

Se quiser limpar após ler:

if (g_view_valid) core_exc_hist_clear();

---

5) Teste prático

1. Compile e rode: g_view_src_name = "NoHistory"
2. Force HardFault (com a função fault_trigger_run_1())
3. O sistema captura, reseta
4. Após reboot:

o g_view_src_name = "HardFault"

o g_view_pc aponta para o local do crash

Resumo — Core ARM Cortex-M0+ (STM32G070RB)

Você já tem a base “de engenheiro” para entender o que acontece antes do main(), como o core executa código e como lidar com interrupções e exceções de forma profissional. Aqui vai um resumo organizado do que cobrimos, para fechar a Parte 2 e abrir caminho para os periféricos da ST com HAL.

---

1) Registradores do core e modos de execução

- R0–R12: uso geral (cálculos, temporários, parâmetros).
- SP: stack pointer. O Cortex-M tem MSP (main) e PSP (process).
  - Bare-metal: normalmente tudo em MSP.
  - RTOS: thread mode tende a usar PSP, handlers usam MSP.
- LR: retorno de função; em exceções vira EXC_RETURN (valor “mágico”).
- PC: próxima instrução.
- xPSR/IPSR: flags e o “número da exceção atual” (ótimo para debug).

---

2) Boot e Reset_Handler (o que acontece antes do main)

- A CPU lê a vector table:

1. word[0] → MSP inicial
2. word[1] → Reset_Handler

- Reset_Handler (startup) tipicamente:
  - copia .data (Flash → RAM)
  - zera .bss
  - chama SystemInit() e runtime (__libc_init_array())
  - chama main()
- Provamos isso com exemplo real: variável em .data chega inicializada e .bss chega zerada.

---

3) Mapa de memória “core-side”

- O core acessa Flash/RAM/periféricos por memória mapeada.
- Periféricos do core (SCB/NVIC/SysTick) vivem no System Control Space (SCS).
- Entender isso ajuda a ler registradores diretamente e a entender por que volatile é obrigatório em registradores.

---

4) Vector table e VTOR

- A vector table contém os endereços dos handlers.
- IRQ externa usa regra: slot = 16 + IRQn.
- VTOR permite realocar a vector table (ex.: para RAM ou para app/bootloader).
- Fizemos exemplos testáveis:
  - ler uma entrada do vetor e comparar com o endereço real do handler
  - copiar vetor para RAM, trocar um handler e apontar VTOR para RAM

---

5) SysTick (timer do core)

- SysTick é um down-counter 24-bit:
  - LOAD = reload, VAL = valor atual, CTRL = enable/irq/clock, CALIB = referência.
- COUNTFLAG indica que ocorreu wrap desde a última leitura (não "conta ticks").
- Configuramos SysTick "na unha" (sem HAL) para 1ms e validamos no debugger.

---

6) NVIC (interrupções externas)

- NVIC controla para cada IRQ externa:
  - Enable (ISER/ICER)
  - Pending (ISPR/ICPR)
  - Priority (IP[])
- No M0+, prioridade é quantizada (poucos bits efetivos).

Ou seja: não existem "256 níveis reais".

- Fizemos exemplos testáveis usando set pending por software para chamar handlers sem periféricos ST.

---

7) SCB (System Control Block) — registradores chave

- ICSR: estado/controle de exceções (VECTACTIVE, VECTPENDING, PENDSTSET/PENDSVSET/NMIPENDSET).
- AIRCR: reset por software (SYSRESETREQ com VECTKEY).
- SCR: comportamento de sleep (SLEEPONEXIT, SLEEPDEEP, SEVONPEND).

- CCR: configurações de alinhamento/controle do core (com ênfase prática no que importa no M0+).
- SHP[]: prioridades de SysTick/PendSV/SVC (system handlers).
- SHCSR: em M0+ é mais limitado; para pending/active, ICSR é o mais útil no dia a dia.

---

8) Exceções: NMI, SVC, PendSV, HardFault

- NMI: não mascarável por PRIMASK. Testamos forçando via ICSR.NMIPENDSET.
- SVC: “trap” via instrução SVC #imm8; extraímos o número do serviço lendo pc_stacked - 2.
- PendSV: exceção pendável usada para trabalho deferido/context switch; testamos via ICSR.PENDSVSET.
- HardFault: no M0+ é o fault principal (não há BusFault/MemManage separados como em M3/M4).

---

9) PRIMASK e seção crítica

- PRIMASK é o mecanismo central no M0+ para travar IRQs mascaráveis.
- Implementamos irq_lock/irq_unlock correto salvando/restaurando PRIMASK.
- Demonstramos “perda” em read-modify-write sem lock e correção com lock.

---

10) Stack frame automático e EXC_RETURN

- Em qualquer exceção, o hardware empilha: r0,r1,r2,r3,r12,lr,pc,xpsr.
- O LR do handler contém EXC_RETURN, que controla como o core retorna da exceção.
- Fizemos demos capturando:

o PC empilhado (ponto interrompido)

o LR empilhado (do código interrompido) vs LR do handler (EXC_RETURN)

o MSP/PSP e CONTROL

---

11) Ferramentas de debug “nível produto”

- Criamos biblioteca para dump de exceção reutilizável.
- Evoluímos para salvar dump em RAM .noinit (persistente após reset).
- Adicionamos:

o CRC/checksum para validar integridade

o ring buffer com histórico dos últimos N faults

o "view" amigável no debugger (strings + valores principais)

Isso vira um "black box recorder" simples e extremamente útil em firmware real.

Desafio do Capítulo 1 — Módulo core-only

Objetivo

Construir um pequeno módulo independente de periféricos da ST para exercitar os conceitos centrais do capítulo. A proposta é unir leitura de registradores do core, uso correto de `PRIMASK`, configuração básica de SysTick e handlers mínimos de exceção.

Requisitos

1. Criar `core_arm.h` e `core_arm.c` com wrappers para:

- ler `IPSR`, `xPSR` e `PRIMASK`
- salvar e restaurar `PRIMASK` em uma seção crítica curta
- configurar o SysTick para gerar um tick periódico simples

2. Criar handlers `NMI_Handler`, `HardFault_Handler` e `SysTick_Handler` com contadores globais.

3. Criar um `main.c` de demonstração que:

- inicializa o módulo
- habilita o tick
- incrementa um contador global no `SysTick_Handler`
- protege ao menos uma atualização compartilhada com `irq_lock()` e `irq_unlock()`

4. Escrever um `startup_notes.md` curto explicando:

- o papel da vector table no reset

- o que o hardware empilha ao entrar em uma exceção
- por que o retorno de handler usa `EXC_RETURN`

Entregáveis esperados

- `core_arm.h`
- `core_arm.c`
- `main.c`
- `startup_notes.md`

Critérios de validação

- `g_systick_ticks` cresce periodicamente
- `PRIMASK` e `IPSR` podem ser observados no debugger
- a seção crítica preserva o valor anterior de `PRIMASK`
- os handlers existem e podem ser localizados na vector table do projeto

Capítulo A — Otimizações do Compilador (GCC) em Firmware

1) O que são otimizações e por que existem

O compilador não "traduz C literalmente". Ele transforma seu código em uma sequência equivalente (do ponto de vista da linguagem) para:

- rodar mais rápido
- ocupar menos flash
- usar menos RAM
- reduzir consumo

Em firmware, isso é crucial: você tem poucos recursos e precisa de determinismo. Mas otimização também expõe erros do código que antes "pareciam funcionar".

---

2) Níveis de otimização do GCC (o que cada um busca)

-O0 (sem otimização)

- Melhor para debug linha a linha

- Gera código maior e mais lento
- Mantém variáveis “visíveis” no debugger

✅ **Use quando:**

- está desenvolvendo e depurando lógica
- quer correspondência forte entre C e assembly

⚠️ **Pegadinha:**

- pode “mascarar” bugs de corrida, volatile faltando e UB.

-Og (otimização amigável para debug)

- Compromisso entre -O0 e -O1
- Mantém debug mais “natural”
- Muitas toolchains recomendam para debug em embedded

✅ **Use quando:**

- quer depurar com algum nível de performance realista

-O1 (otimização leve)

- Remove redundâncias simples
- Inlines moderados
- Código geralmente melhor sem “mudanças agressivas”

-O2 (otimização forte para performance)

- Mais inlining
- Melhor reordenação e eliminação de código morto
- Pode mudar bastante a estrutura do código no disassembly

✅ **Use quando:**

- performance importa
- você já tem testes e estabilidade

⚠️ **Mais chance de:**

- variáveis “sumirem” no debug
- bugs de UB aparecerem

-O3 (otimização agressiva)

- Ainda mais inlining e otimizações de loop
- Pode aumentar código e piorar cache (em MCUs sem cache, ainda pode aumentar flash e afetar timing)

✅ **Em embedded, -O3 nem sempre é melhor. Muitas vezes -O2 ou -Os é preferível.**

-Os (otimização para tamanho)

- Tenta reduzir tamanho do binário
- Excelente para MCUs com flash limitada
- Pode reduzir inlining e gerar código mais compacto

✅ **Muito usado em produto.**

---

3) O que o compilador faz na prática (transformações comuns)

- Dead code elimination: remove código que nunca afeta saída
- Constant folding: pré-calcula expressões constantes
- Common subexpression elimination: evita recalcular mesma coisa
- Loop optimizations: unroll, strength reduction
- Inlining: substitui chamada por corpo (reduz overhead, aumenta flash)
- Reordering: reordena instruções, desde que a linguagem permita

---

4) Por que bugs “somem” ou “aparecem” com otimização

Aqui estão os campeões em firmware.

4.1 Variável não inicializada

Em -O0 o layout e os padrões de stack às vezes fazem “parecer” que funcionou.

Em -O2, registradores e stack mudam e o bug explode.

✅ **Regra: sempre inicialize variáveis.**

4.2 Falta de volatile em flags de ISR/DMA

Sem volatile, o compilador pode:

- ler uma vez e guardar em registrador
- não reler nunca mais
- transformar loops em infinito

Exemplo típico:

```
while (!flag) { }
```

Sem volatile, em -O2 pode virar:

- "flag não muda" (na visão do compilador) → loop infinito

✅ **Regra:**

- flag modificada por ISR/DMA → volatile

4.3 Race conditions (concorrência)

Em -O0, o código é mais lento e "muda o timing", então a corrida pode não acontecer.

Em -O2, o timing muda e a corrida aparece.

✅ **Regra:**

- proteger seções críticas (PRIMASK/RTOS)
- usar estruturas corretas (fila circular, buffers)
- evitar read-modify-write sem proteção em variáveis compartilhadas

---

4.4 Undefined Behavior (UB) — o "vilão" da otimização

Se o seu código entra em comportamento indefinido, o compilador pode gerar qualquer coisa.

O exemplo mais comum em embedded:

- overflow de signed
- ponteiro inválido

- acesso fora do array

Em -O0 pode “passar”, em -O2 pode quebrar.

✅ **Regras práticas:**

- evite overflow signed (use int32_t com cuidado, ou uint32_t quando apropriado)
- valide índices
- evite casts perigosos

---

5) Pegadinhas famosas com otimização (e como evitar)

5.1 static inline e side-effects

Inlining pode duplicar código e avaliar expressões de forma diferente.

✅ **Regra: cuidado com macros e funções inline com parâmetros que têm efeitos colaterais.**

5.2 Funções “sumindo” no debug

Em -O2, funções podem ser inline e “não existir” como frame de stack.

✅ **Dica:**

- use -Og para debug
- ou marque função com __attribute__((noinline)) temporariamente

---

5.3 Otimização vs tamanho (trade-off real)

- -O2 pode aumentar flash (mais inlining)
- -Os pode reduzir flash mas mudar performance

✅ **Em produto, normalmente você escolhe:**

- -Os para dispositivos apertados
- -O2 para performance crítica

---

6) Como depurar bugs relacionados à otimização

Checklist rápido:

1. Compare -O0 vs -Og vs -O2

2. Se bug aparece só com otimização:

o suspeite de volatile faltando

o suspeite de UB

o suspeite de concorrência

3. Use o disassembly para confirmar:

o leituras de flags continuam existindo?

o loops foram eliminados?

4. Ative warnings: -Wall -Wextra -Werror

5. Se possível, use sanitizers em host (quando o código puder ser compilado no PC)

---

7) Recomendações STM32CubeIDE

- Debug: -Og
- Release: -Os (geral) ou -O2 (performance)
- Sempre revisar:

o volatile em registradores/flags ISR

o seções críticas

o inicialização

Capítulo B — Linker Script em Firmware (o que é e como fazer do zero)

1) O que é um linker script e por que ele importa

Depois de compilar, você tem arquivos .o com:

- código (.text)
- constantes (.rodata)

- variáveis inicializadas (.data)
- variáveis zero (.bss)
- símbolos e relocations

O linker é quem decide:

- onde cada seção vai ficar na memória
- qual é o endereço de cada símbolo
- como gerar o .elf/.bin

Em microcontrolador, isso é crítico porque:

- Flash e RAM têm endereços fixos
- stack e heap precisam caber
- vetor de interrupções deve estar no endereço certo
- bootloader/app podem dividir flash

---

2) Memória típica de um STM32 (conceito)

- FLASH: código e constantes
- RAM: .data, .bss, stack, heap
- vector table: normalmente no início da flash (ou remapeada)

---

3) Seções principais (as que você precisa dominar)

- .isr_vector → tabela de vetores
- .text → código
- .rodata → constantes
- .data → globais inicializadas (copia Flash → RAM no reset)
- .bss → globais não inicializadas (zeradas no reset)
- .stack (às vezes implícito) → stack
- .heap (às vezes implícito) → heap
- .noinit → RAM persistente (não zera) — que já usamos

---

4) Como o Reset_Handler usa símbolos do linker script

O startup precisa saber:

- onde começa .data na Flash
- onde começa .data na RAM
- tamanho de .data
- onde começa e termina .bss

Esses endereços vêm de símbolos gerados pelo linker script (ex.: _sidata, _sdata, _edata, _sbss, _ebss).

✅ **Sem esses símbolos, não existe "cópia de data" nem "zerar bss".**

---

5) Linker script do zero (modelo didático)

A seguir é um "esqueleto" de linker script para um MCU com:

- FLASH em 0x08000000 (típico STM32)
- RAM em 0x20000000

No seu projeto real, você ajusta os tamanhos conforme o STM32G070RB.

Exemplo de .ld (didático)

```
/* ====== Definição de memória ====== */

MEMORY

{

  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 128K

  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 36K

}


/* ====== Símbolos úteis ====== */

_estack = ORIGIN(RAM) + LENGTH(RAM);  /* topo da stack */


/* ====== Seções ====== */

SECTIONS
```

```
{
  /* Vetor de interrupções no início da FLASH */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector))
    . = ALIGN(4);
  } > FLASH

  /* Código e constantes */
  .text :
  {
    . = ALIGN(4);
    *(.text)
    *(.text*)
    *(.rodata)
    *(.rodata*)
    . = ALIGN(4);
    _etext = .;
  } > FLASH

  /* Dados inicializados: LMA em FLASH, VMA em RAM */
  _sidata = LOADADDR(.data);

  .data : AT ( _etext )
  {
    . = ALIGN(4);
    _sdata = .;
```

```
  *(.data)
  *(.data*)
  . = ALIGN(4);
  _edata = .;
} > RAM

/* BSS: zerado no Reset_Handler */
.bss :
{
  . = ALIGN(4);
  _sbss = .;
  *(.bss)
  *(.bss*)
  *(COMMON)
  . = ALIGN(4);
  _ebss = .;
} > RAM

/* NOINIT: RAM persistente */
.noinit (NOLOAD) :
{
  . = ALIGN(4);
  _snoinit = .;
  *(.noinit)
  *(.noinit*)
  . = ALIGN(4);
  _enoinit = .;
} > RAM
```

}

---

6) Como validar o linker script (sem adivinhação)

Ferramentas práticas:

- arquivo .map (mostra onde cada símbolo caiu)
- no debugger: observar endereços de variáveis
- confirmar que:

o .isr_vector está no início da flash

o .data está na RAM e tem LMA em flash

o .bss está na RAM

o .noinit fica na RAM e não é zerada

---

7) Pegadinhas comuns com linker script

- colocar .isr_vector fora do início → MCU não dá boot
- errar tamanho de RAM → stack invade .bss e crash aleatório
- esquecer KEEP no vetor → linker pode remover vetor “não referenciado”
- .data sem AT ou sem _sidata → variáveis inicializadas vêm erradas
- misturar .noinit com .bss → perde persistência